

PACKET CAPTURE: OVERVIEW AND BASICS

Byeong-hee Roh

Dept. of Software and Computer Engineering

Ajou University



변화와 융합의 중심
MMcN

멀티미디어 융합 연구실

Mobile Multimedia convergent Network Lab.
<http://mmcn.ajou.ac.kr>



Contents

- ❖ Sockets
- ❖ Packet Capture Basics
- ❖ Understanding Libpcap and Npcap
- ❖ Pcap Data Structure
- ❖ Example – Captured File Analysis

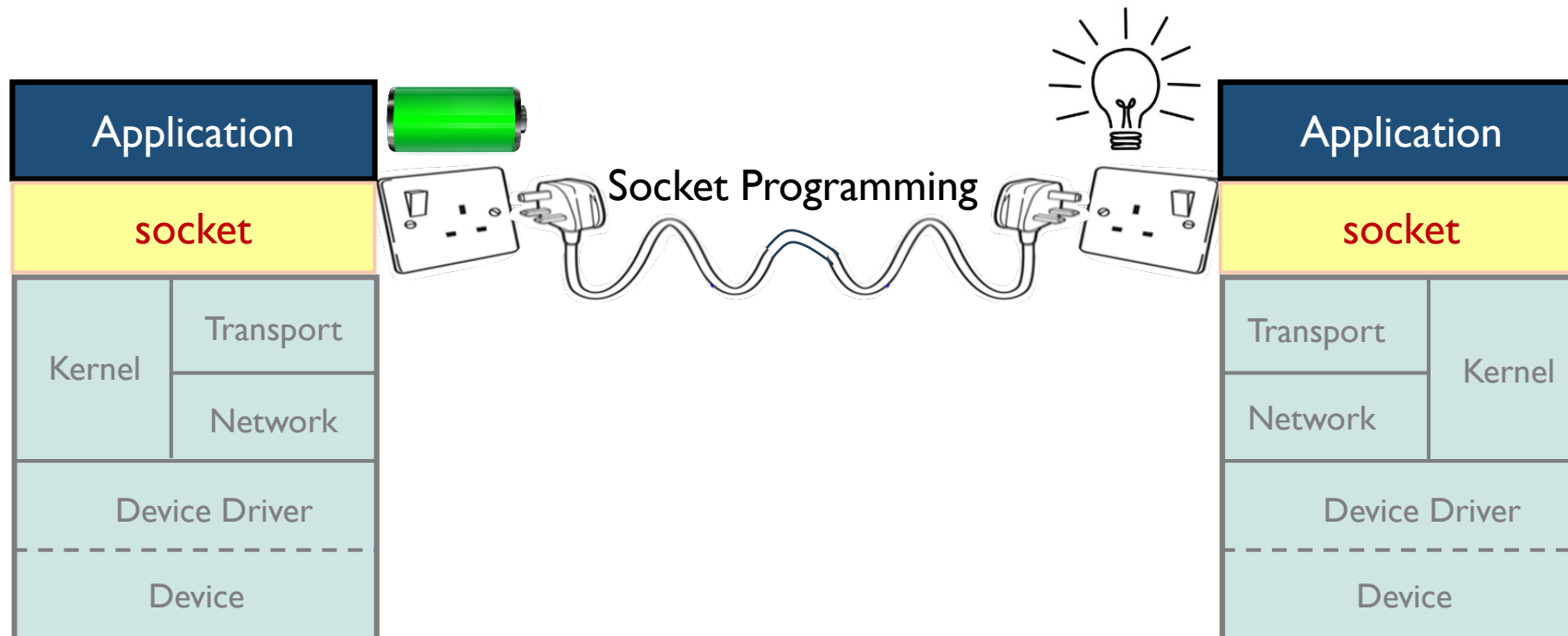


SOCKETS

Network Application Programming

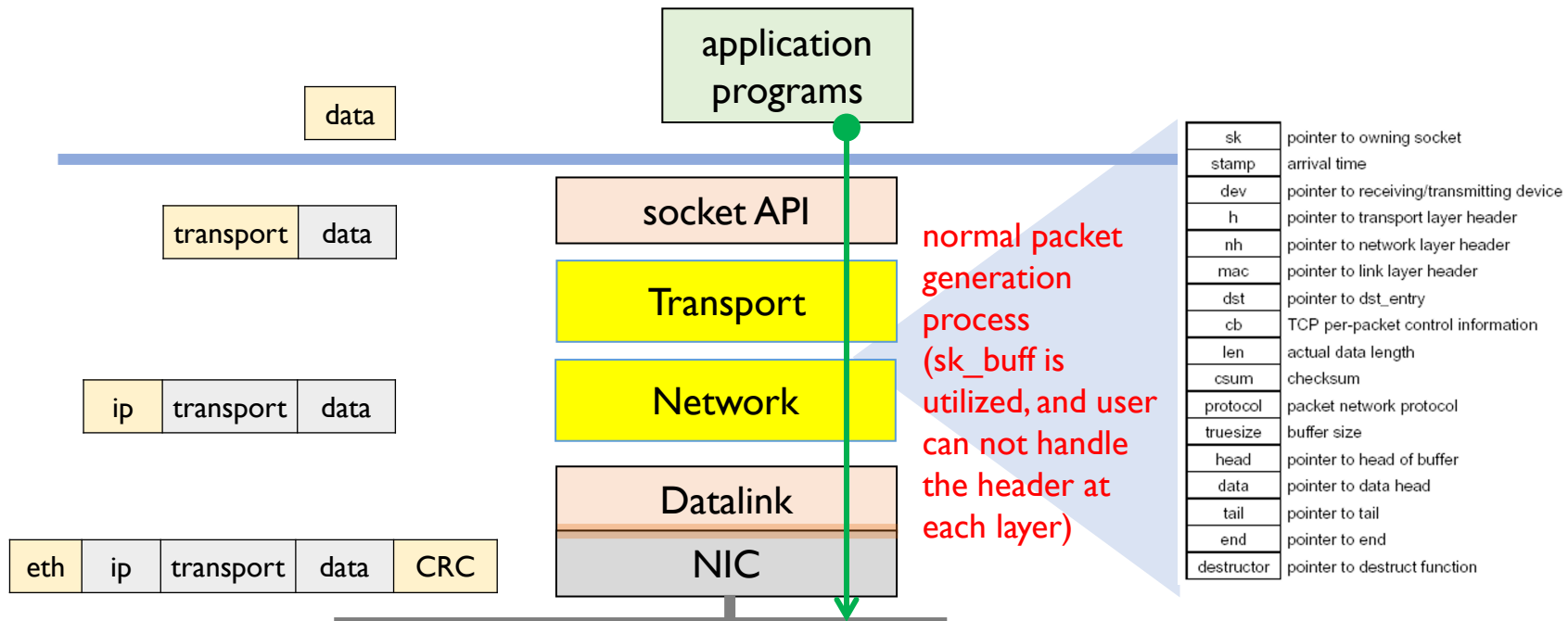
❖ Sockets provide

- network programming interface between
 - processes at application layers on the network
 - through TCP/IP network kernel (OS), but don't care of it



Standard Socket (1/3)

- ❖ In standard sockets,
 - the SDU from upper layer is encapsulated at a layer
 - TCP/UDP, IP, and Frame headers are automatically added, and
 - user don't need to aware the existence of TCP/IP layer

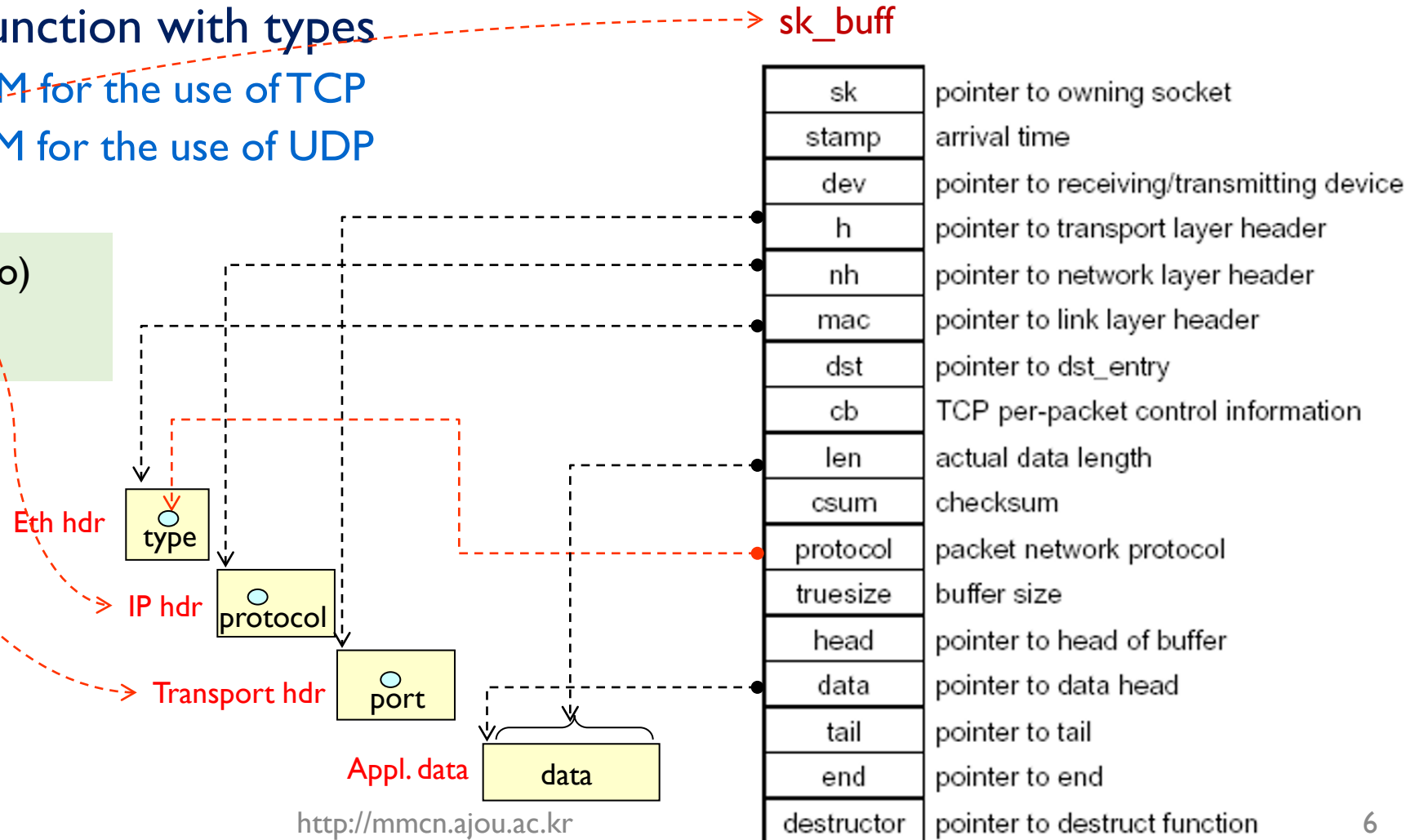


Standard Socket (2/3)

❖ To create standard sockets

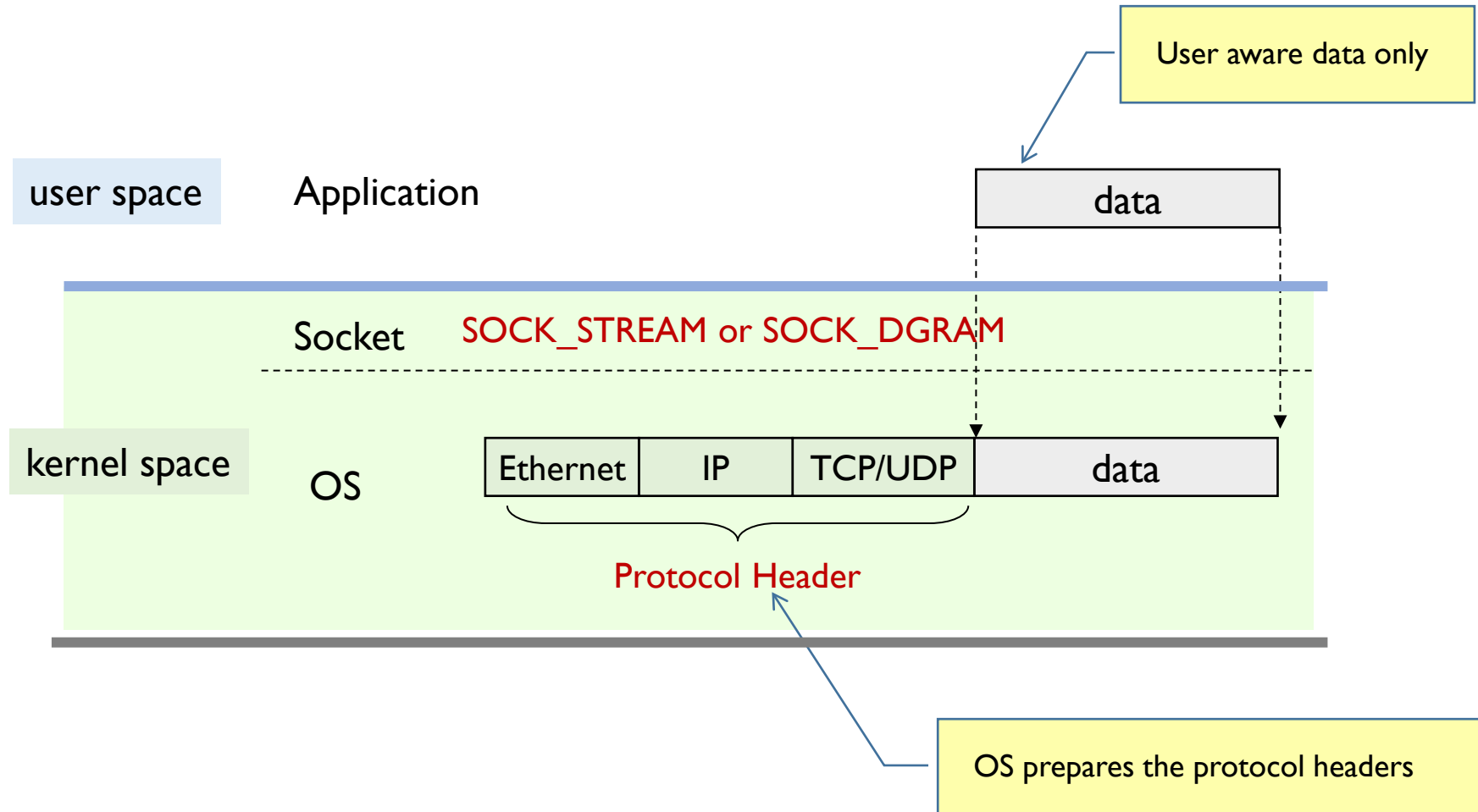
- use `socket( )` function with types
 - `SOCK_STREAM` for the use of TCP
 - `SOCK_DGRAM` for the use of UDP

```
sd = socket (dm, type, proto)
z = bind (sd, addr, len)
```



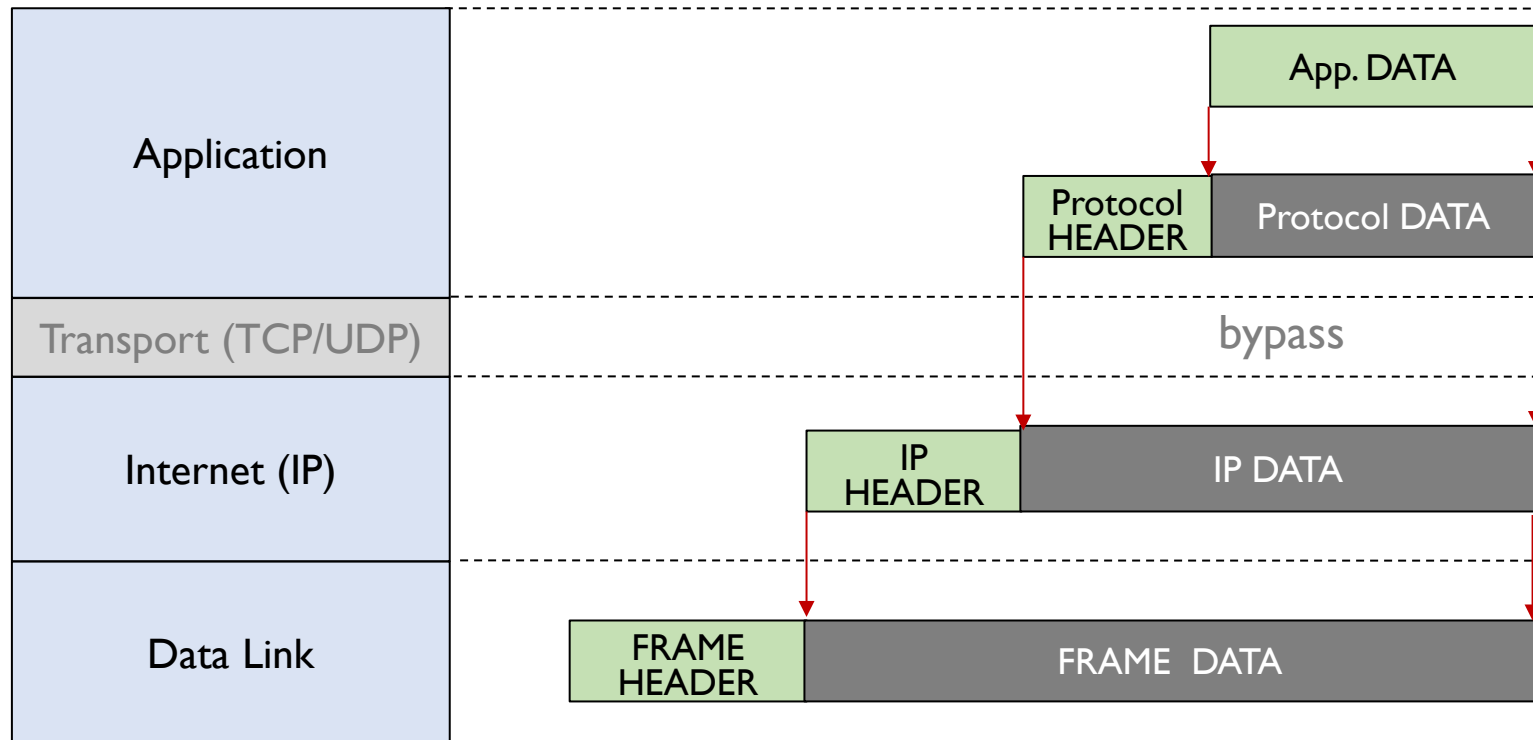
Standard Socket (3/3)

❖ SOCK_STREAM and SOCK_DGRAM



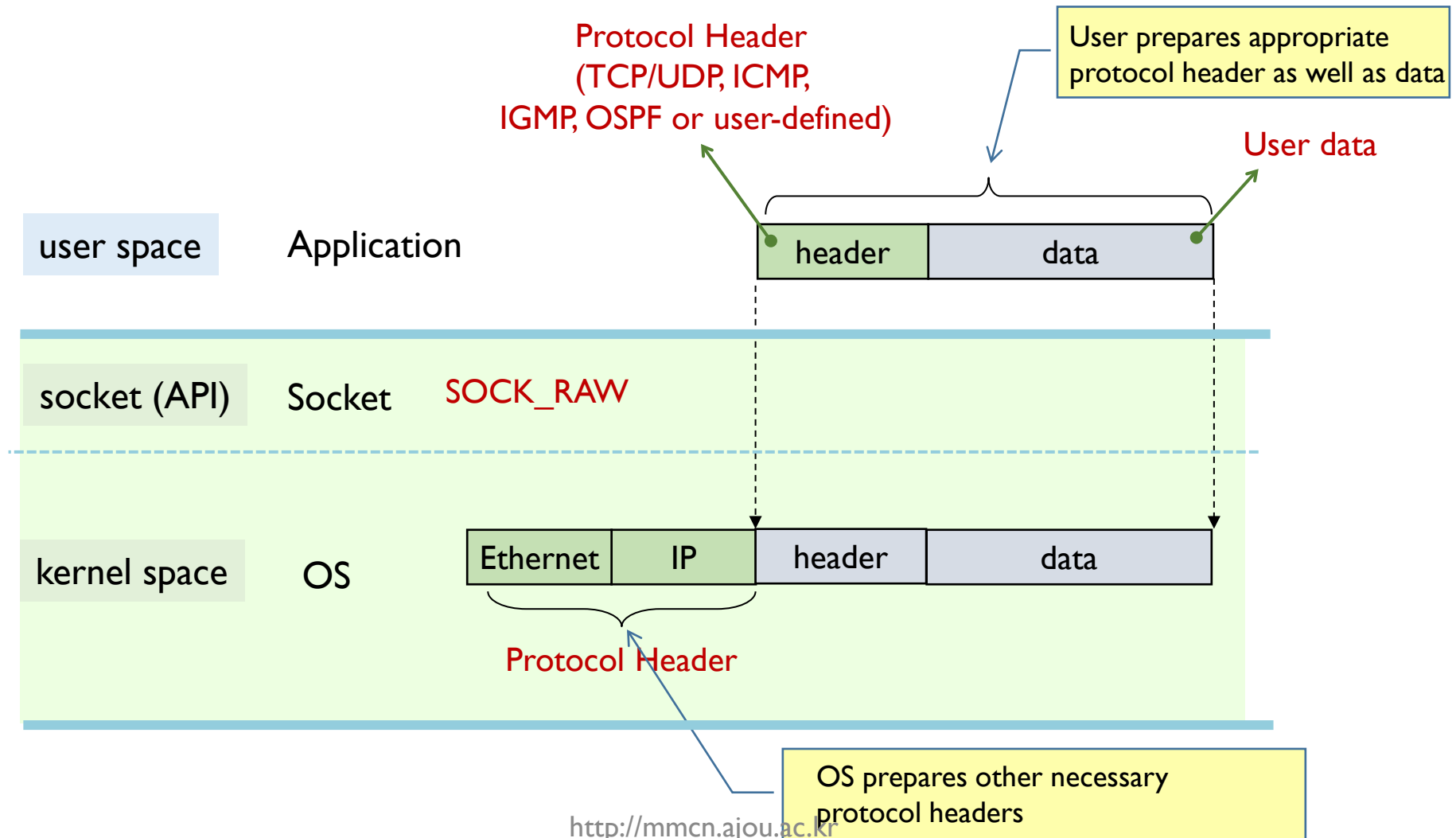
Raw Socket (1/2)

- ❖ A **raw socket** is
 - a type of socket that allows **bypass the transport layer**
 - user should prepare IP data part with appropriate headers and data



Raw Socket (2/2)

❖ RAW Socket

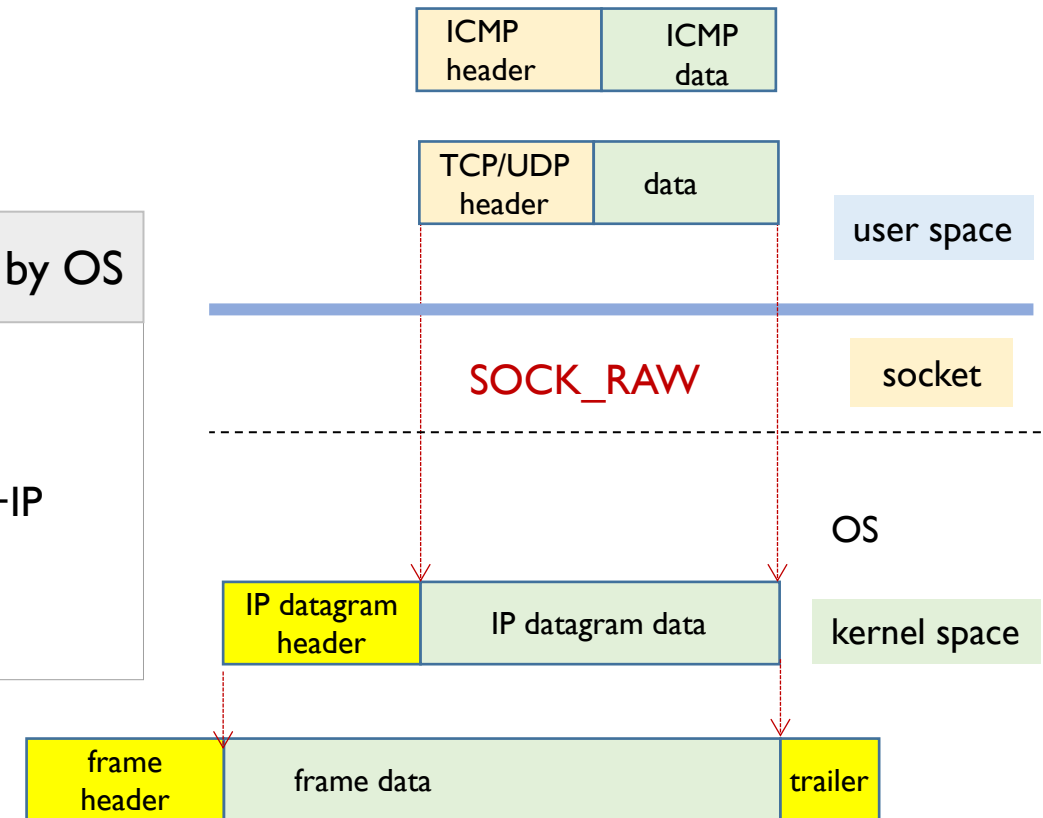


RAW Socket with IP_HDRINCL option (1/3)

❖ SOCK_RAW **without** IP_HDRINCL option

- it bypasses Transport layer only
- OS prepares IP header at network layer

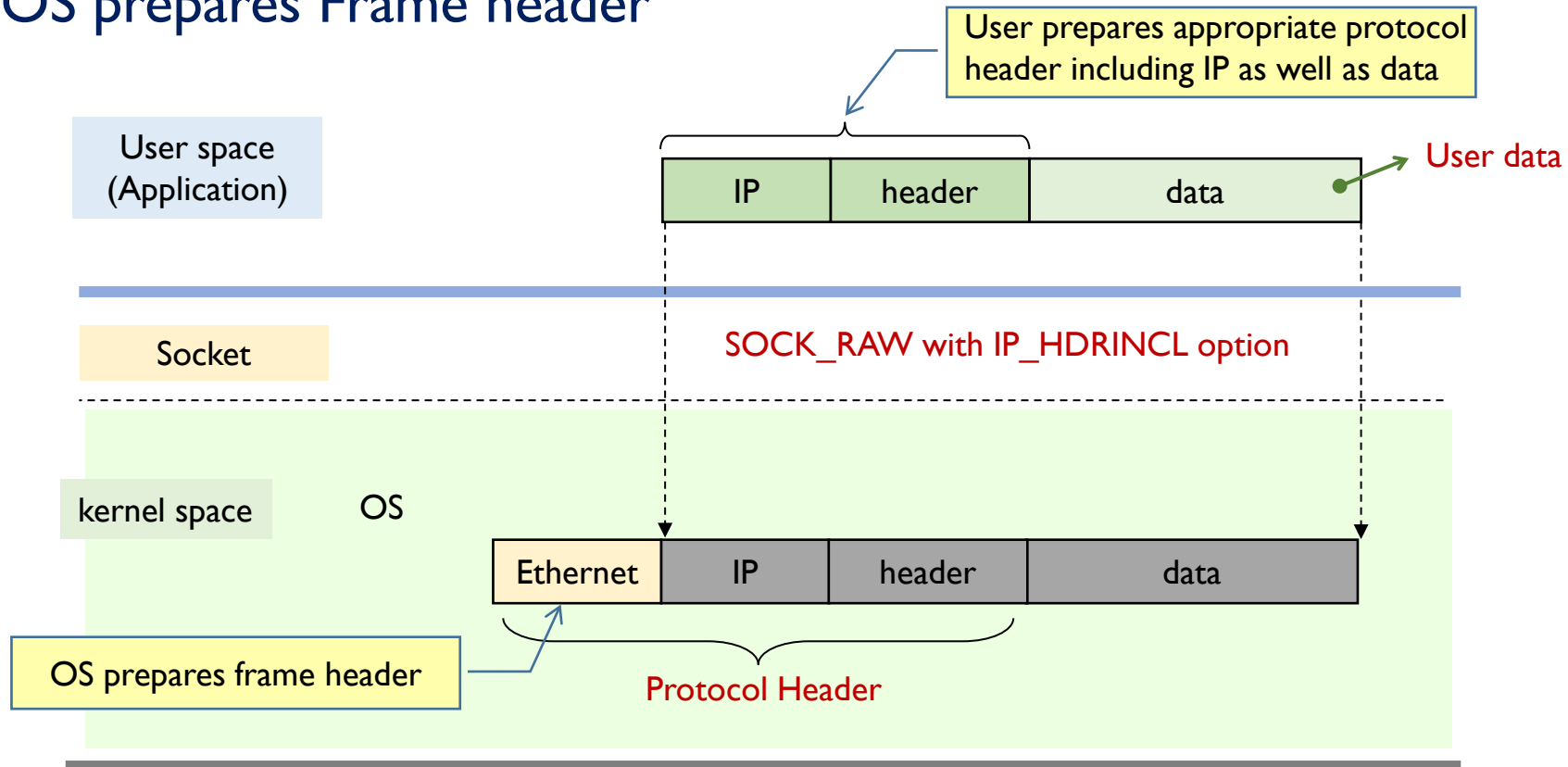
Protocol	Header given by Application	Header by OS
IPPROTO_ICMP	ICMP (ICMP Header + ICMP Data)	Eth+IP
IPPROTO_IGMP	IGMP (IGMP Header + IGMP Data)	
IPPROTO_TCP	TCP (TCP Header + App. Data)	
IPPROTO_UDP	UDP (UDP Header + App. Data)	
user defined	user defined header (IP + USER)	



RAW Socket with IP_HDRINCL option (2/3)

❖ SOCK_RAW with IP_HDRINCL option

- it bypasses both transport and IP layer
- OS prepares Frame header





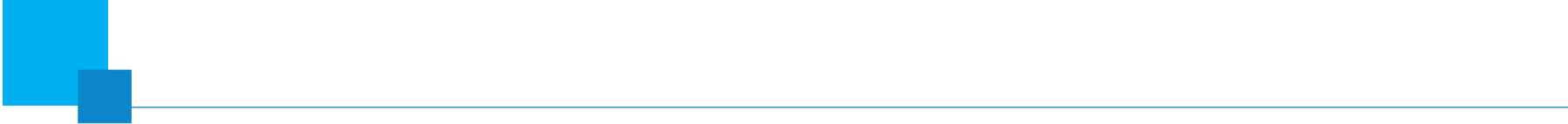
RAW Socket with IP_HDRINCL option (3/3)

❖ Limitations of IP_HDRINCL option

- IP_HDRINCL only allows for creating IP headers **on outgoing** packets
 - it does not provide any functionality for capturing or analyzing incoming traffic
 - so, it can't be used for packet analysis
- IP_HDRINCL requires IP header by the application for each packet sent
 - it does not inherently handle lower layers, such as the Ethernet layer
- IP_HDRINCL depends on system-specific behavior
 - Some OSs may not fully support or implement this option in a consistent way
 - Windows does not support TCP packet transmission with IP_HDRINCL option

❖ libpcap

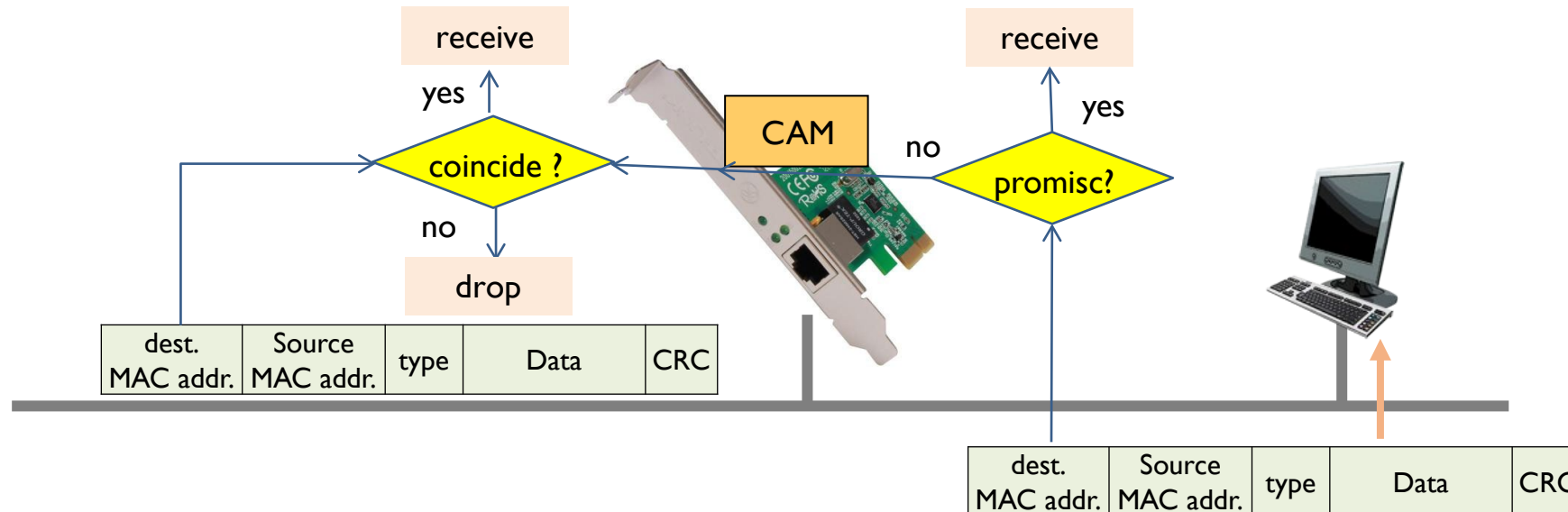
- preferable and comprehensive packet capture and analysis library
 - compared to IP_HDRINCL, which is more specialized and limited to sending custom IP packets only



PACKET CAPTURE BASICS

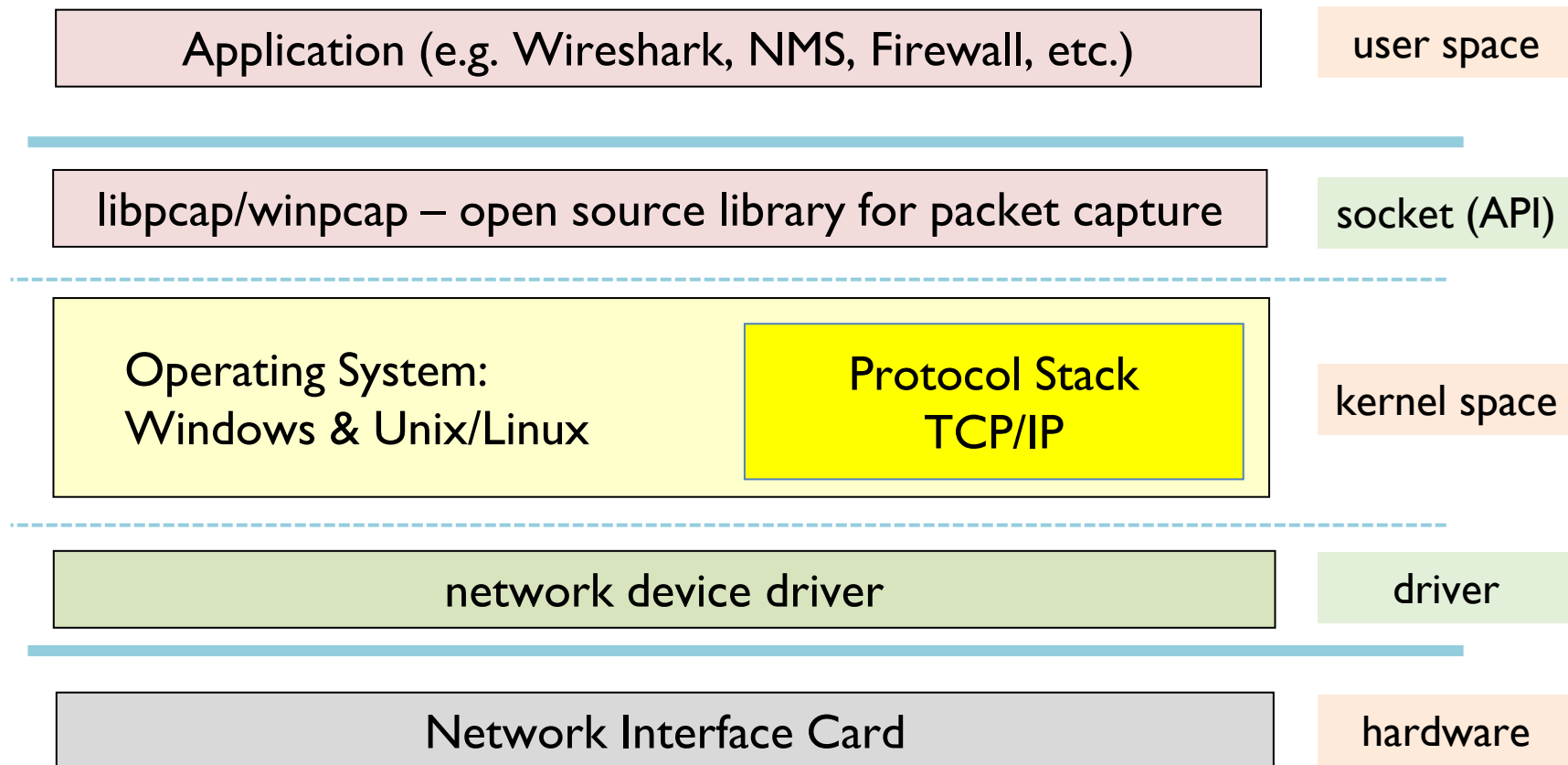
Network Interface Card (NIC)

- ❖ NIC accepts frames with their destination addresses
 - its MAC address – stored in EEPROM
 - multicast addresses – stored in Content Addressable Memory(CAM), and
 - broadcast address
- ❖ Promiscuous Mode
 - NIC accepts all frames it receives



Packet Capture Structure

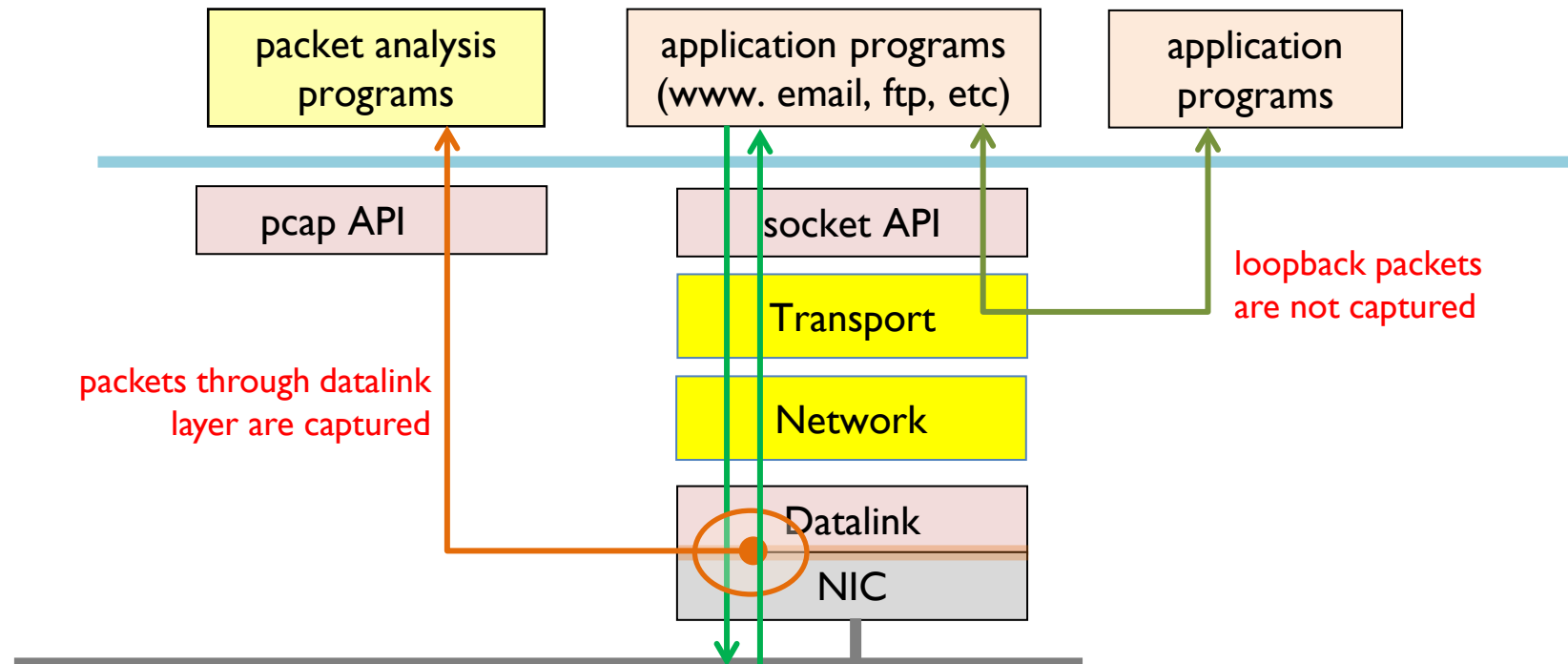
❖ Wireshark, OS, and Pcap library



Packet Capture Structure

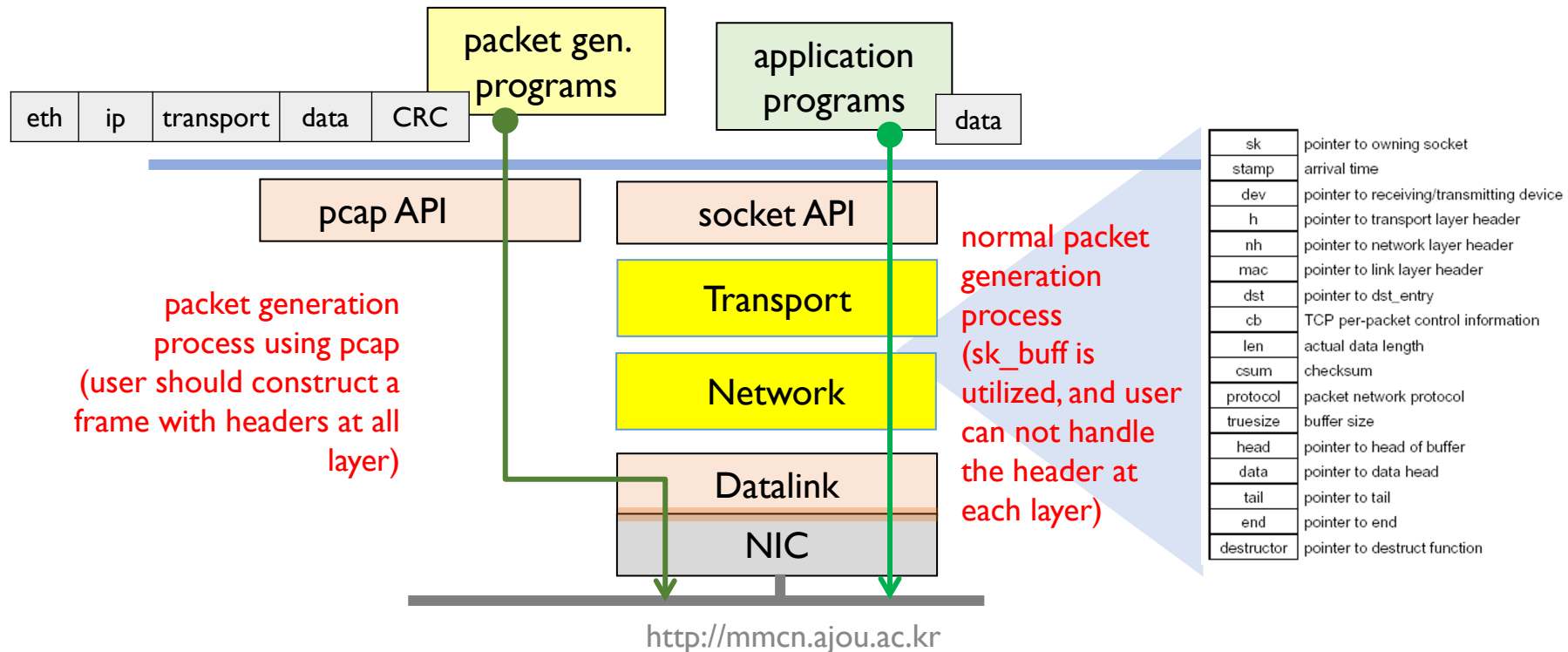
❖ Packet Capture Layer

- Packets are **not captured at all protocol layers**,
- Packets **in and out through NIC (data link layer)** are captured.
 - with pcap library, frames received and transmitted at datalink layer are delivered to application programs by passing tcp/ip stack



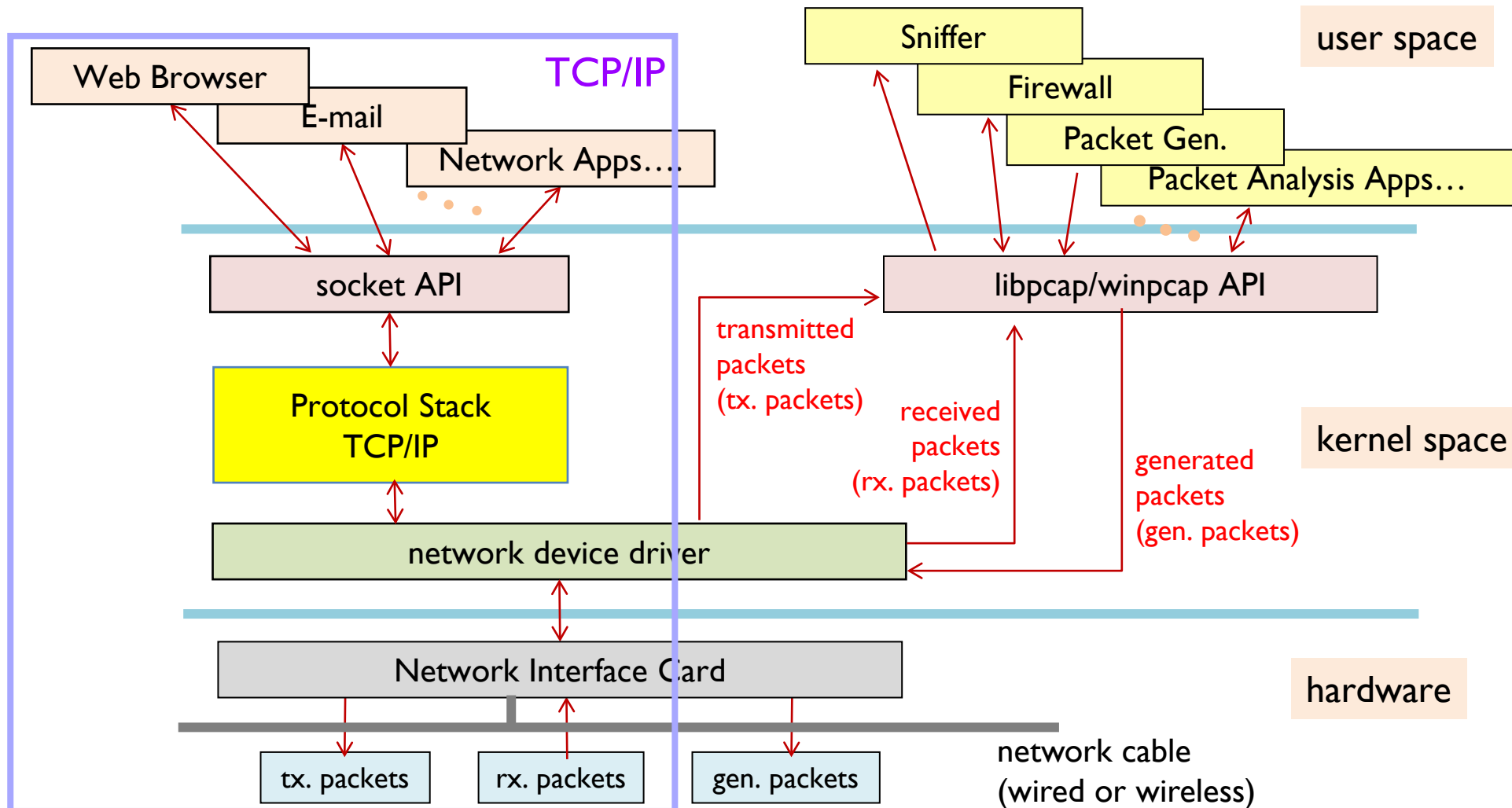
Packet Generation Structure

- ❖ pcap API provides packet generation process
 - an application program
 - constructs a full frame format with headers at all layers including application data, then
 - sends it through NIC directly using pcap library



Packet Analysis Process

❖ Packet sending, capture, and generation



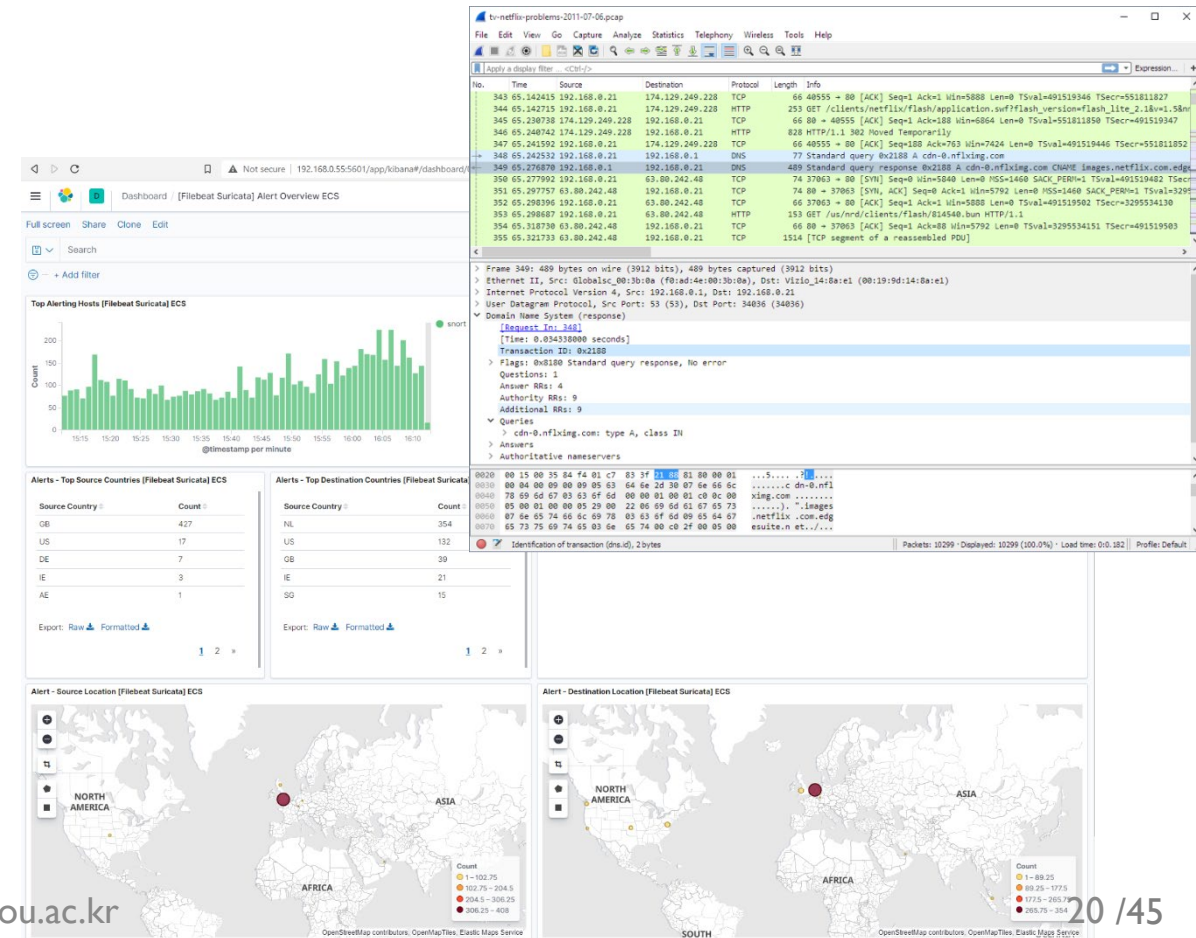
UNDERSTANDING LIBPCAP AND NPCAP



pcap library

❖ pcap (packet capture)

- open source library providing application programming interface (API) for packet-capture and network analysis
- written in C/C++ basically
- capable of
 - to capture network packets
 - to save captured packets
 - to analyze the real-time and saved packets
 - to inject packets, and so on



libpcap on Linux (1/2)

❖ libpcap on Linux

- developed in 1994
 - by the tcpdump developers in the Network Research Group at Lawrence Berkeley Laboratory
 - <http://www.tcpdump.org>
 - <https://github.com/the-tcpdump-group>
- installed by default on the BSDs, macOS, and Ubuntu Linux
 - Latest Releases
 - tcpdump: version: 4.99.5, released on Aug. 2024
 - libpcap: version 1.10.5, released on Aug. 2024
 - Checking the installation of libpcap



```
tcpdump --version
```

- ex)

```
bhroh@mmcn:~$ tcpdump --version
tcpdump version 4.99.4
libpcap version 1.10.4 (with TPACKET_V3)
OpenSSL 3.0.13 30 Jan 2024
```



libpcap on Linux (2/2)

- ❖ libpcap development package: libpcap-dev
 - includes the header files and development tools
 - needed to use the libpcap library in C or C++ programs
 - allows programs to access libpcap functionality
 - by providing files such as pcap.h
 - note: libpcap itself is a packet capture library used for capturing and analyzing network traffic. If the libpcap-dev package is not installed, pcap.h will be missing, leading to compilation errors when libpcap functions are used.
 - install the libpcap-dev

```
sudo apt update
sudo apt install libpcap-dev
```



Npcap on Windows

❖ WinPcap

- the Windows version of libpcap
 - <https://www.winpcap.org/>
- still available, but has not been maintained or updated since 2013

❖ Npcap

- an architecture for packet capture and network analysis for Windows OS
 - all of WinPcap's packet capture and injection features are included,
 - with a few great additions like raw 802.11 frame capture
- Npcap facilities
 - capture raw packets
 - filter the packets according to user-specified rules
 - transmit raw packets to the network
 - gather statistical information on the network traffic engine
 - support for remote packet capture.

Npcap on Windows

❖ Comparisons of features for Npcap and WinPcap

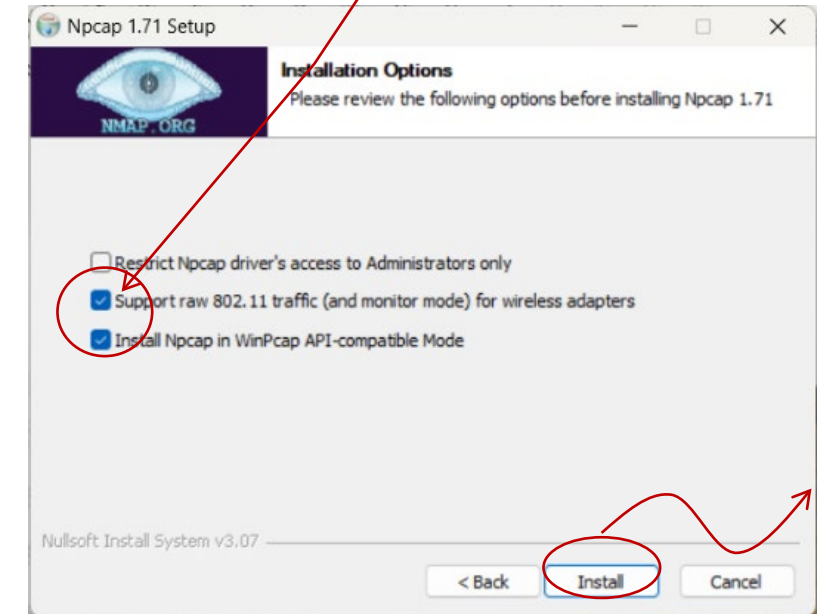
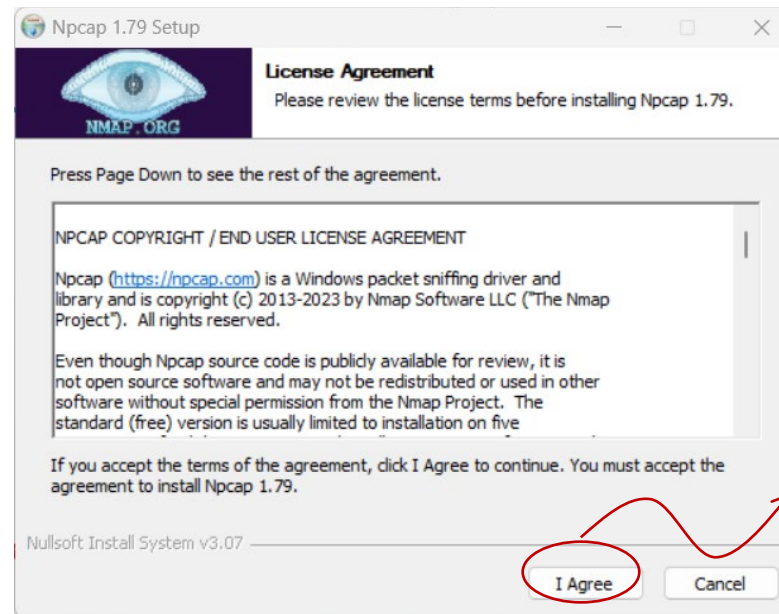
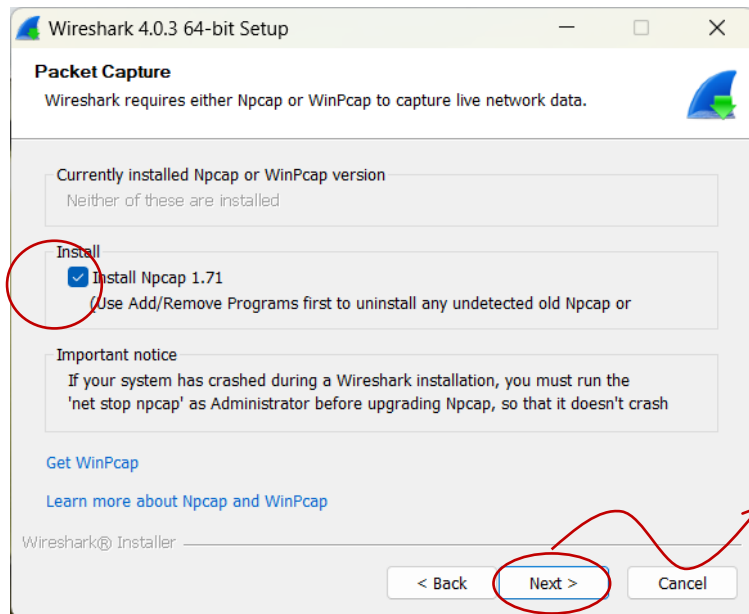
	Feature	Npcap	WinPcap
information	Actively maintained and supported	Yes	No (terminated)
	Latest release	Sep. 16, 2024	Mar. 8, 2013
	libpcap version	1.10.4 (2023)	1.0.0 (2008)
	License	Free for personal use	BSD-style
	Supported commercial/redistributable version	Yes (Npcap OEM)	No (terminated)
security	EV SHA-256 code signing	Yes	No
	Limit access to administrators (optional)	Yes	No
Basic Features	Packet capture with the cross-platform libpcap API	Yes	Yes
	Link-layer packet injection	Yes	Yes
	Source code available	Yes	Yes
Advanced Features	Capture raw 802.11 frames	Yes (with many adapters)	Yes (with specialized AirPcap)
	Capture Loopback traffic	Yes	No
	Inject Loopback traffic	Yes	No

Npcap on Windows

❖ install Npcap

- install during Wireshark installation
- download Npcap installer at <https://npcap.com/#download>
 - Npcap 1.80 installer for Windows 7, 8, 8.1, 10, 11 (x86, x64 and ARM64)
 - Latest development source: Github (<https://github.com/nmap/npcap>)

only for 802.11 WiFi packet capture, but the wifi adaptor should support monitor mode. Find adaptors in monitor mode at https://secwiki.org/w/Npcap/WiFi_adapters



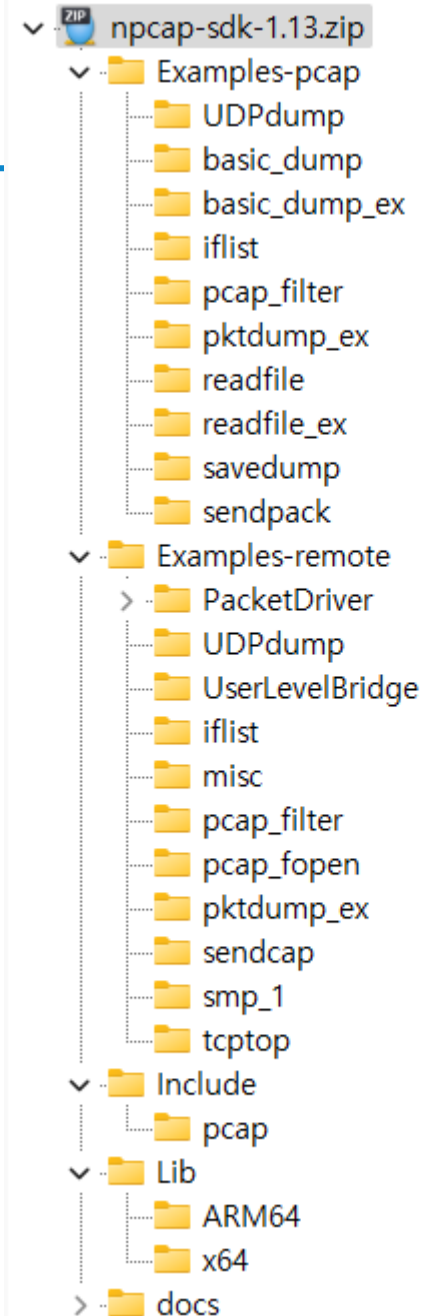
Npcap Programming

❖ Npcap SDK

- download Npcap SDK
 - at <https://npcap.com/#download>
 - Npcap SDK 1.13 (ZIP)
- unpack the SDK

❖ Resources

- Development Tutorial
 - <https://nmap.org/npcap/guide/npcap-tutorial.html>
- User's Guide
 - <https://nmap.org/npcap/guide/>
- WiFi adaptors in monitor mode
 - https://secwiki.org/w/Npcap/WiFi_adapters
- Comparisons to WinPcap
 - <https://nmap.org/npcap/vs-winpcap.html>



Npcap local capture
application example codes

Npcap remote capture
application example codes

Libraries and Include files
for Npcap programming

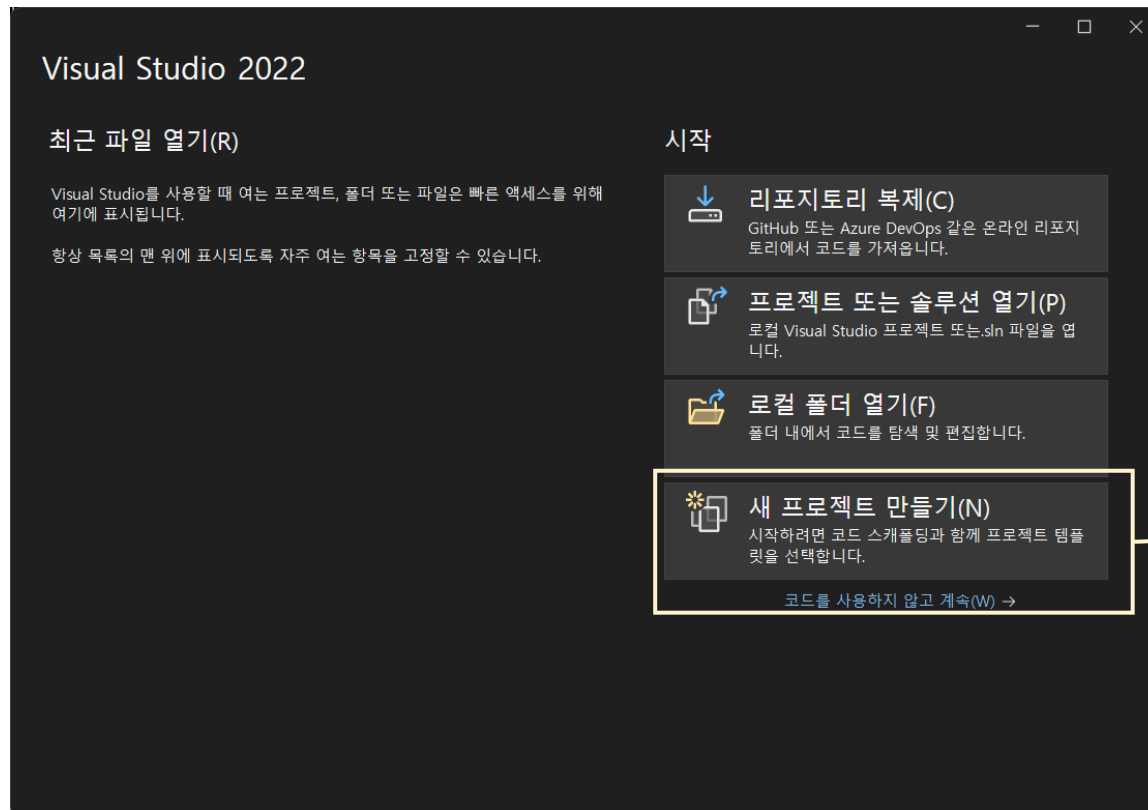


Npcap Programming with Visual Studio

- ❖ Configuring MS Visual Studio compilation environments
 - Include the file <pcap.h> at the beginning of every source file
 - that uses the functions exported by library.
 - Add directories for include and library files, for example
 - NpcapSDK\include for include directory
 - NpcapSDK \lib for library directory
 - Include the following preprocessor definitions
 - WPCAP for Win32 specific functions of Npcap
 - HAVE_REMOTE for the remote capture capabilities of Npcap
 - Set the options of the linker to include the wpcap.lib
 - Set the options of the linker to include the winsock library file ws2_32.lib.

Npcap Programming with Visual Studio

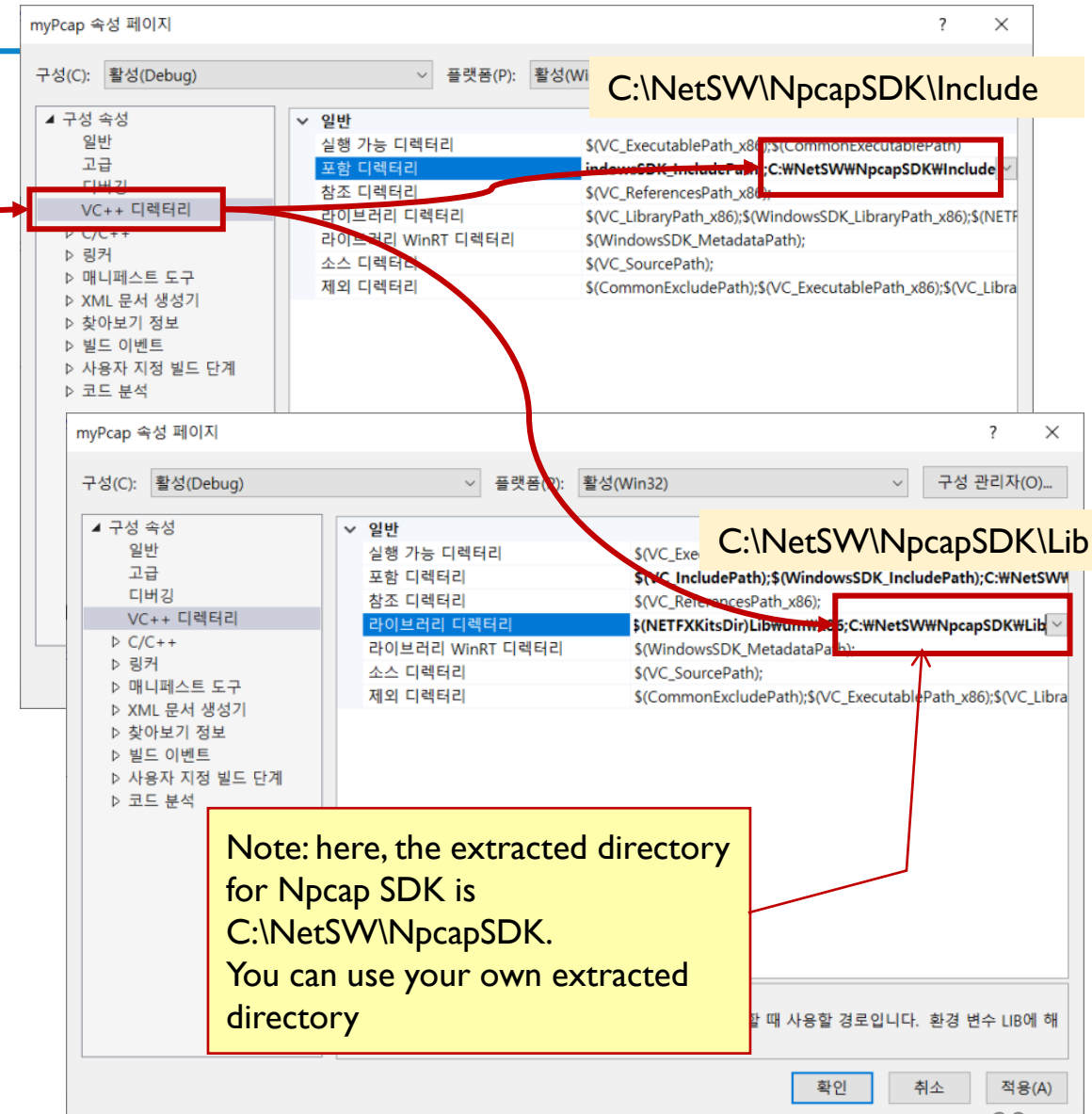
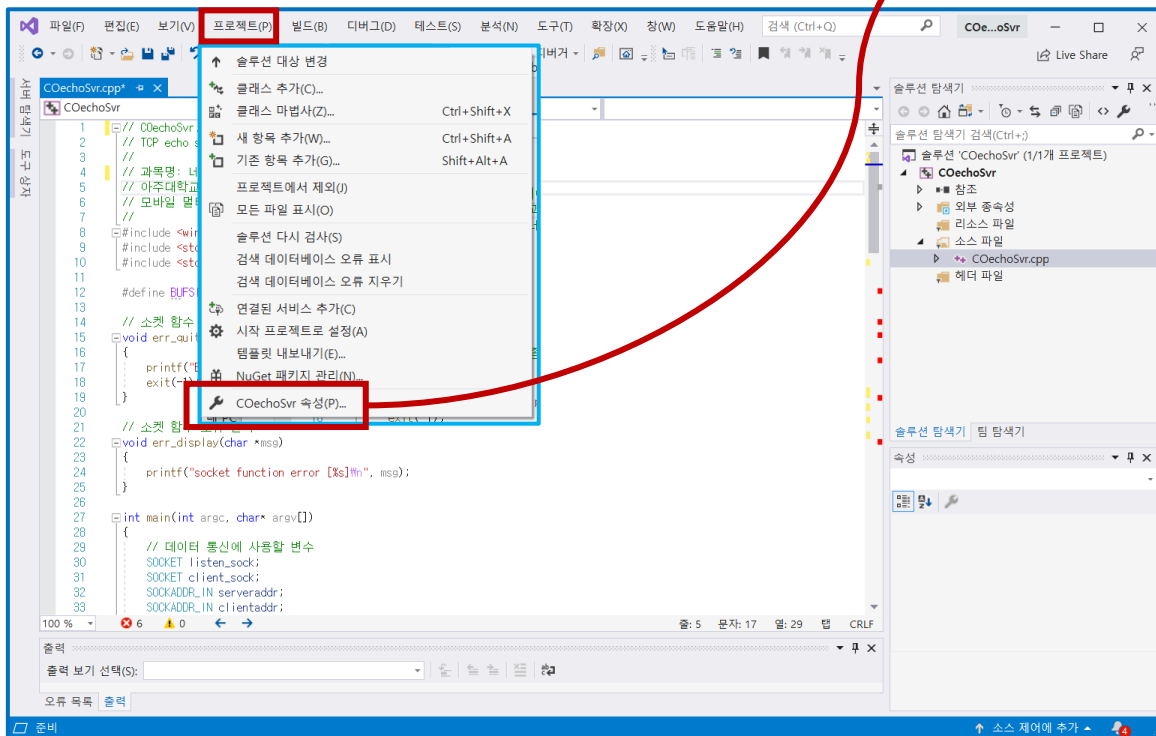
- ❖ Create a project for a Npcap programming
 - Microsoft Visual Studio



Npcap Programming with Visual Studio

❖ Library and Include Files

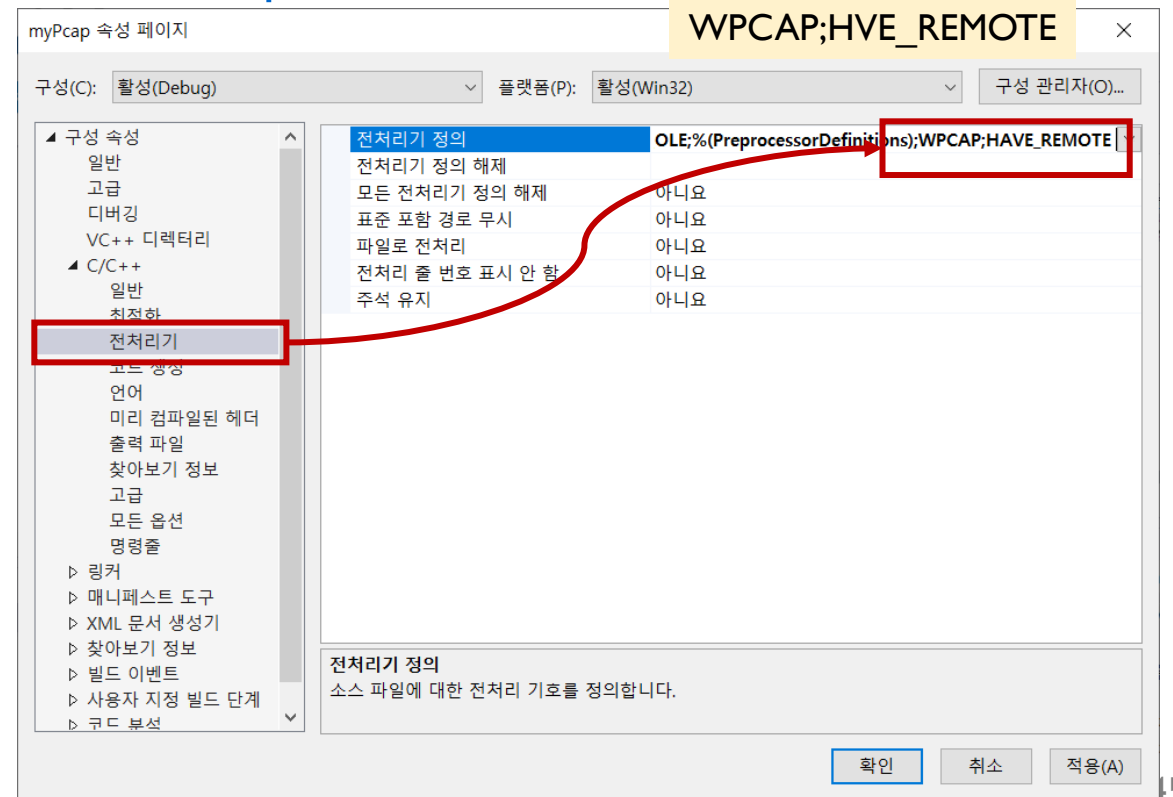
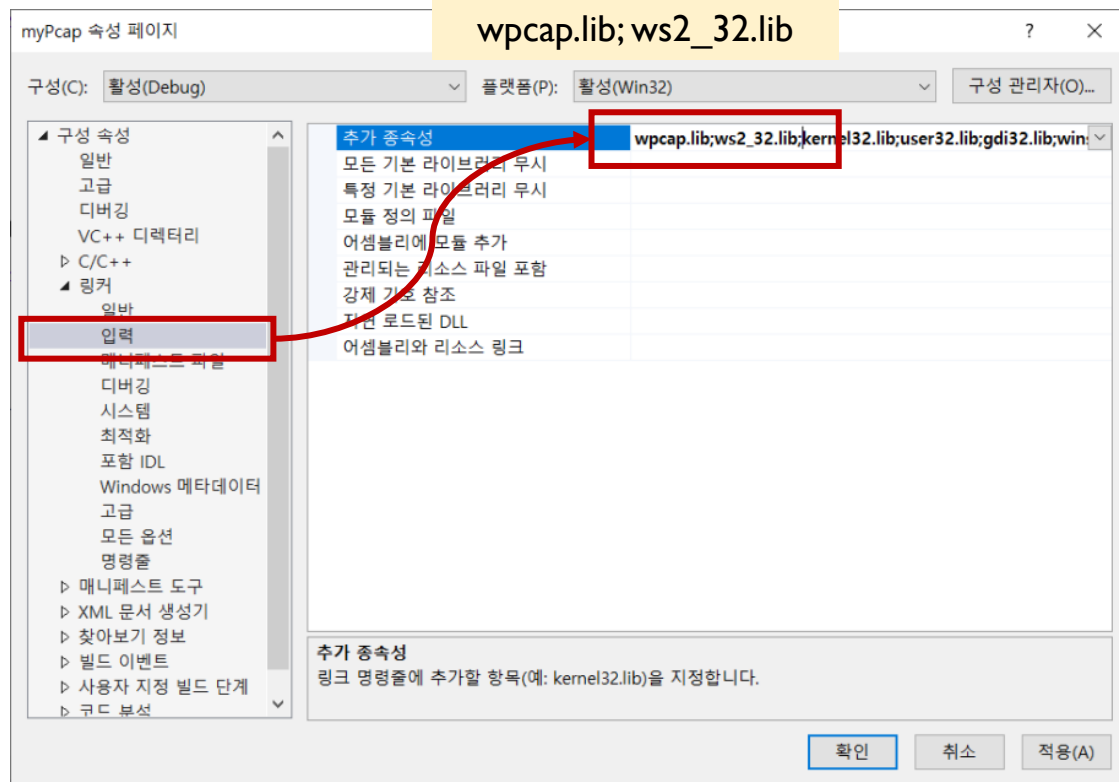
- add directories for include and library files



Npcap Programming with Visual Studio

❖ Set the linker options

- add wpcap.lib and ws2_32.lib
- define preprocessors
 - WPCAP for local and HAVE_REMOTE for remote captures





Npcap Functions

- ❖ Obtaining the list and advanced information about devices
- ❖ Capturing the packets from devices
- ❖ Capturing the packets without the callback
- ❖ Filtering the traffic
- ❖ Interpreting the packets
- ❖ Handling offline dump files
- ❖ Sending Packets
- ❖ Gathering Statistics on the network traffic

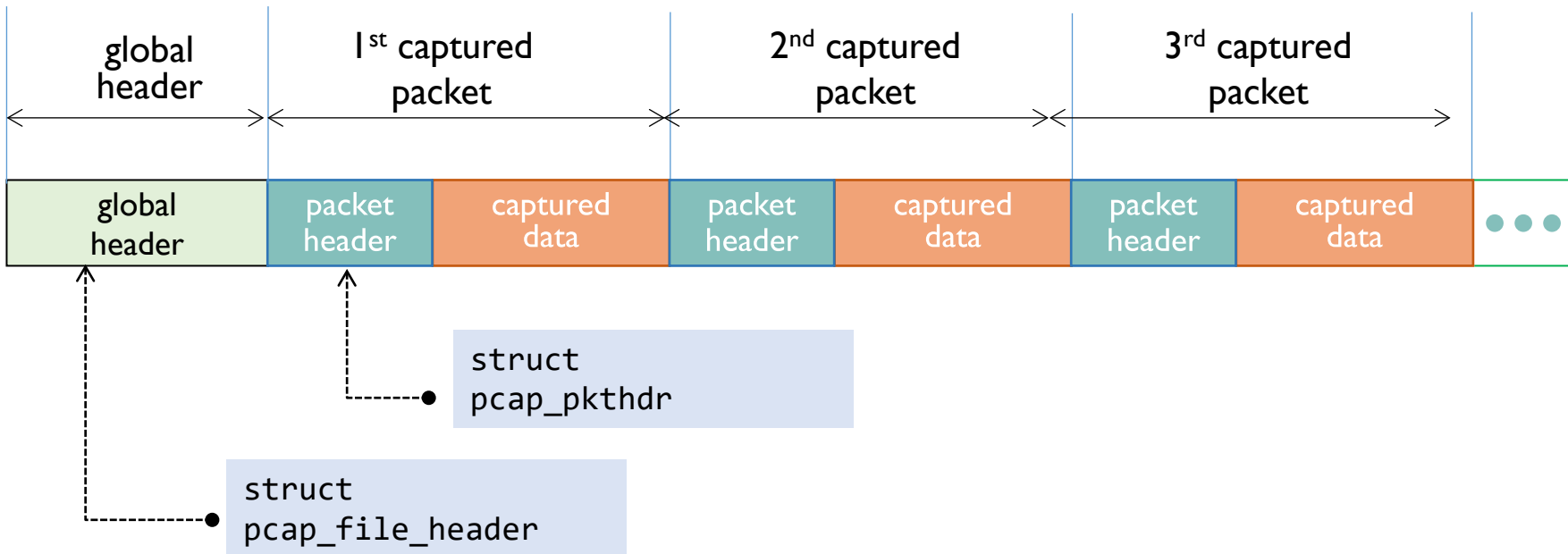


PCAP DATA STRUCTURE

Pcap Data Structure

❖ Data structure of captured file

- begins with global header, and
- repeats each captured packet with header and data



Pcap Data Structure

❖ Global header (24 bytes)

- It contains metadata about the capture
- defined in <pcap.h>

```
struct pcap_file_header {  
    bpf_u_int32      magic;           // 0xA1B2C3D4 for a valid pcap file  
    u_short          version_major;  
    u_short          version_minor;  
    bpf_int32        thiszone;       // gmt to local correction - not used  
    bpf_u_int32      sigfigs;        // accuracy of timestamps - not used  
    bpf_u_int32      snaplen;        // max length saved portion of each pkt  
    bpf_u_int32      linktype;       // data link type (e.g. Ethernet, 802.11, ..)  
};
```

▪ macros

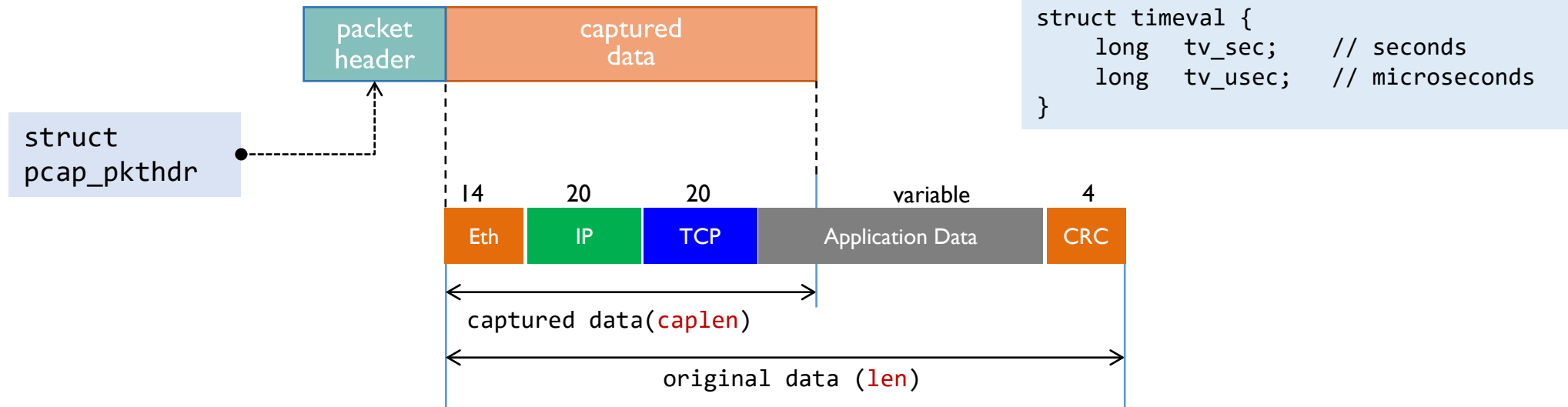
```
#define u_int32      bpf_u_int32  
#define int32        bpf_int32
```

Pcap Data Structure

❖ Packet header (16 bytes)

- it contains the time and lengths information of each captured packet

```
struct pcap_pkthdr {  
    struct timeval    ts;        // time stamp (seconds and microseconds)  
    bpf_u_int32       caplen;    // length of captured packet (bytes)  
    bpf_u_int32       len;       // total length of the original packet (bytes)  
};
```



```
struct timeval {  
    long    tv_sec;    // seconds  
    long    tv_usec;   // microseconds  
}
```

Wireshark Packet Capture Options

❖ Packet Capture Option - Input

capture Options

capture interface의 input 옵션 설정

caplen 지정 (1~262144)

WiFi packet capture in monitor mode

모든 패킷들을 캡처하도록 함 (미 설정시, 자신의 송수신 패킷들만 캡처됨)

특정 패킷들만 캡처하도록 지정함

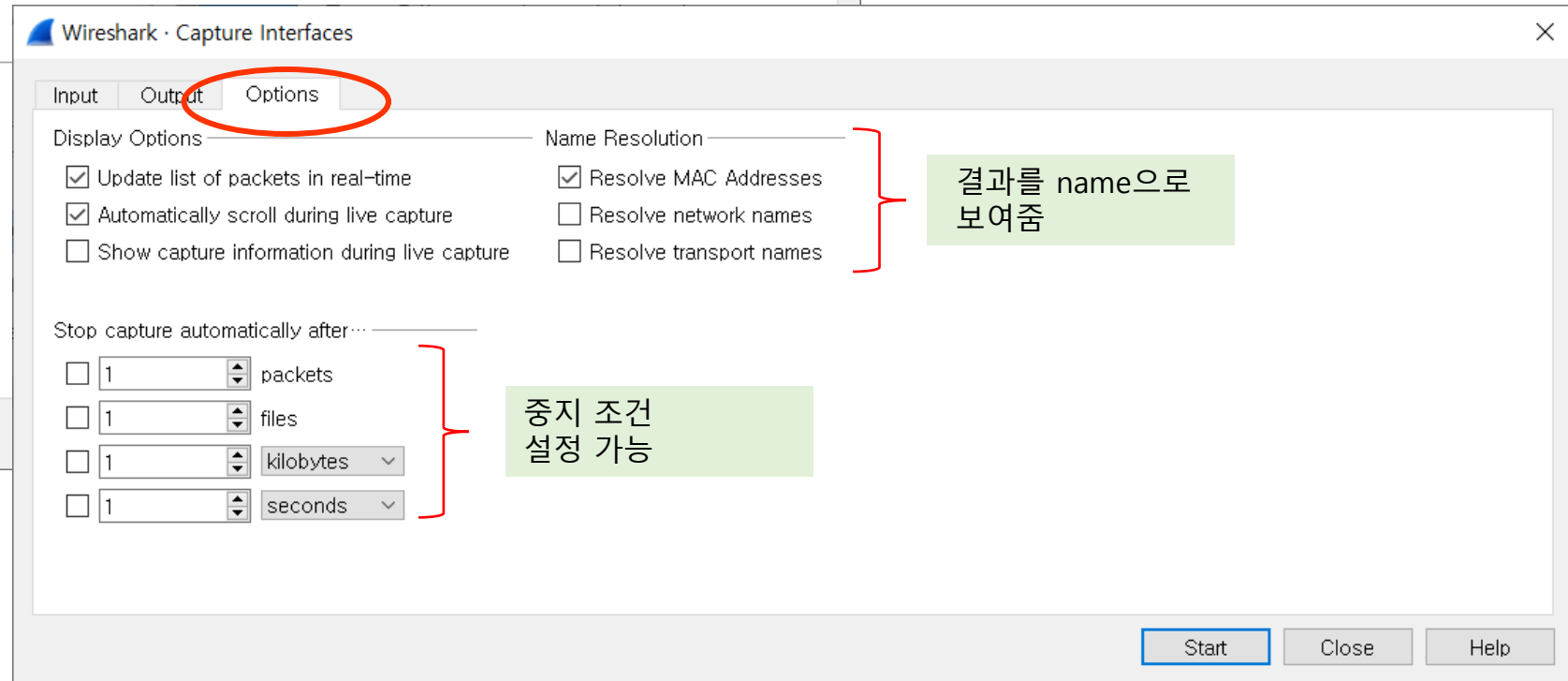
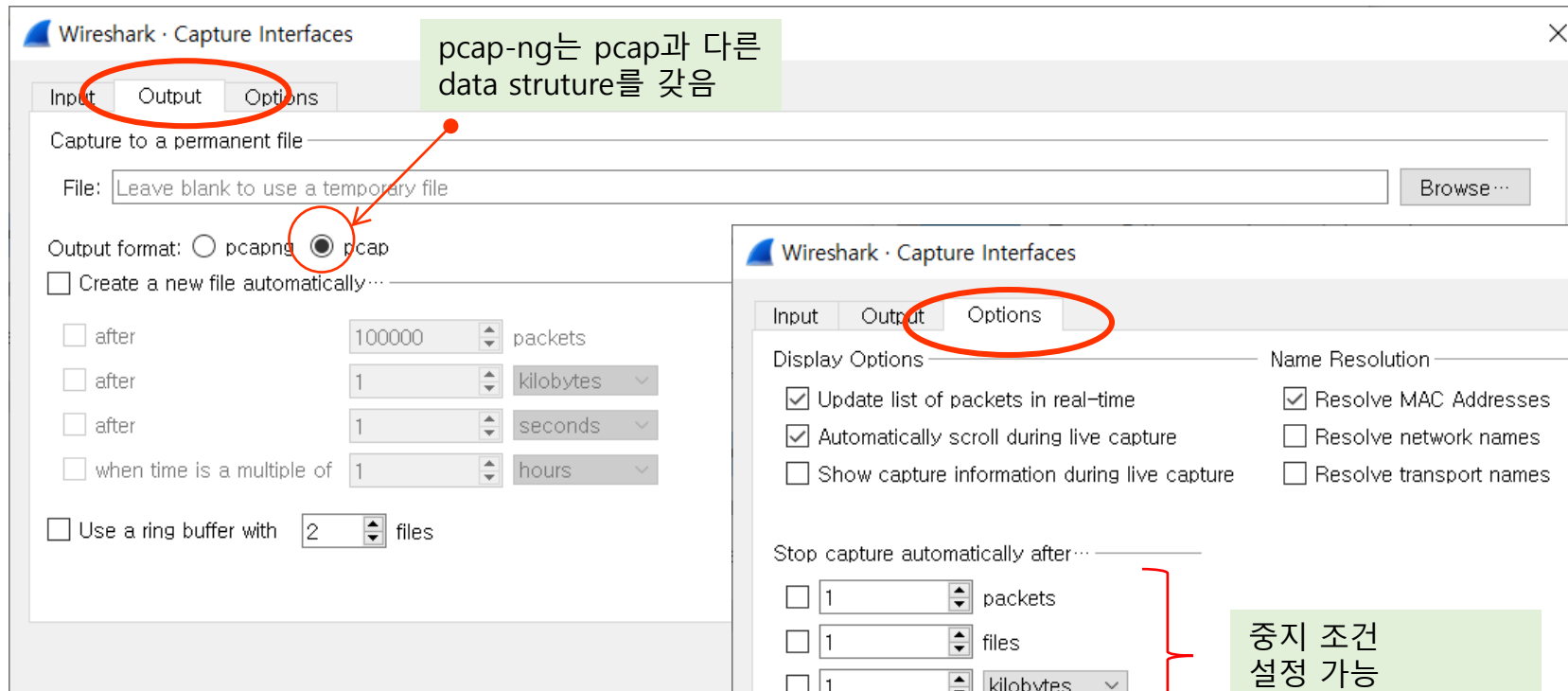
The image shows the Wireshark Packet Capture Options dialog box. The 'Input' tab is selected. The 'Interface' list shows various network interfaces. The 'Traffic' column shows the type of traffic captured. The 'Link-layer Head' column shows the link-layer head type. The 'Promiscuous' column shows whether promiscuous mode is enabled. The 'Snaplen (B)' column shows the snaplen value. The 'Buffer (MB)' column shows the buffer size. The 'Monitor Mode' column shows whether monitor mode is enabled. The 'Capture Filter' column shows the capture filter. The 'Enable promiscuous mode on all interfaces' checkbox is checked. The 'Capture filter for selected interfaces' field contains 'tcp port http'. The 'Start' button is highlighted.

Interface	Traffic	Link-layer Head	Promiscuous	Snaplen (B)	Buffer (MB)	Monitor Mode	Capture Filter
로컬 영역 연결* 9		Ethernet	☒	262144	2	☐	
> 이더넷		Ethernet	☒	default	2	☐	
> Wi-Fi		Ethernet	☒	default	2	☐	
로컬 영역 연결* 10		Ethernet	☒	default	2	☐	
로컬 영역 연결* 2		Ethernet	☒	default	2	☐	
> 이더넷 2		Ethernet	☒	default	2	☐	
로컬 영역 연결* 1		Ethernet	☒	default	2	☐	
Adapter for loopback traffic capture		BSD loopback	☒	default	2	☐	
로컬 영역 연결* 8		Ethernet	☒	default	2	☐	
USBPcap1		USBPcap	☐	—	—	☐	

tcp port http

Wireshark Packet Capture Options

❖ Packet Capture Option – Output and Options





PCAP PROGRAMMING EXAMPLE – CAPTURED FILE ANALYSIS



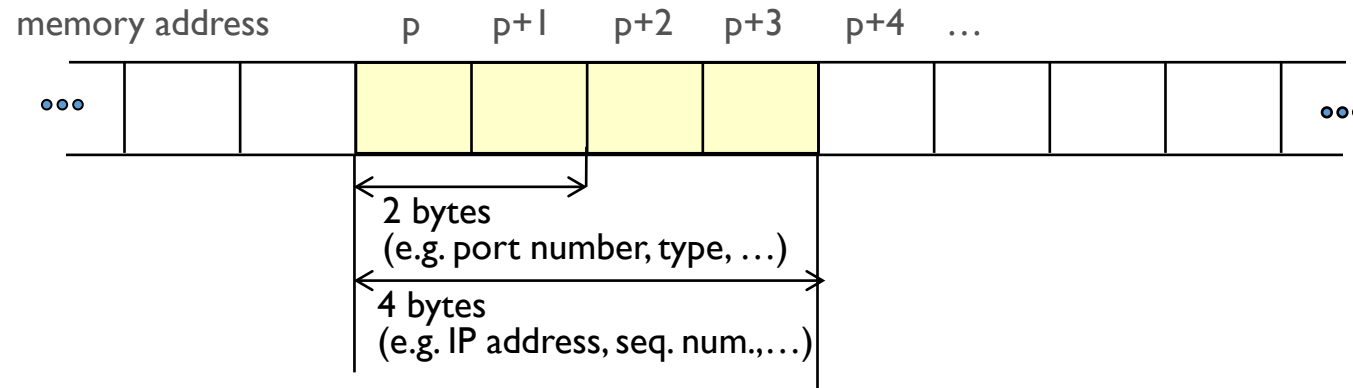
Captured File Analysis

❖ Procedure

- Step 1 : run WireShark
 - set appropriate parameters
- Step 2 : capture packets and store them to a file
 - run some networking applications such as ftp, ping, http, ...
- Step 3 : analyze capture file using capture file analysis program
 - open the capture file
 - read each packet according to pcap data structure
 - analyze each followings:
 - Ethernet frame header
 - IP datagram header
 - TCP/UDP header
 - Application protocol header or data
 - analyze more
 - connection information
 - application usage (time, number of exchanged packets, ...)

Tips on writing programs

❖ Reading a field with 2 or 4 bytes from a given memory address



- macro to read a value with 2 bytes

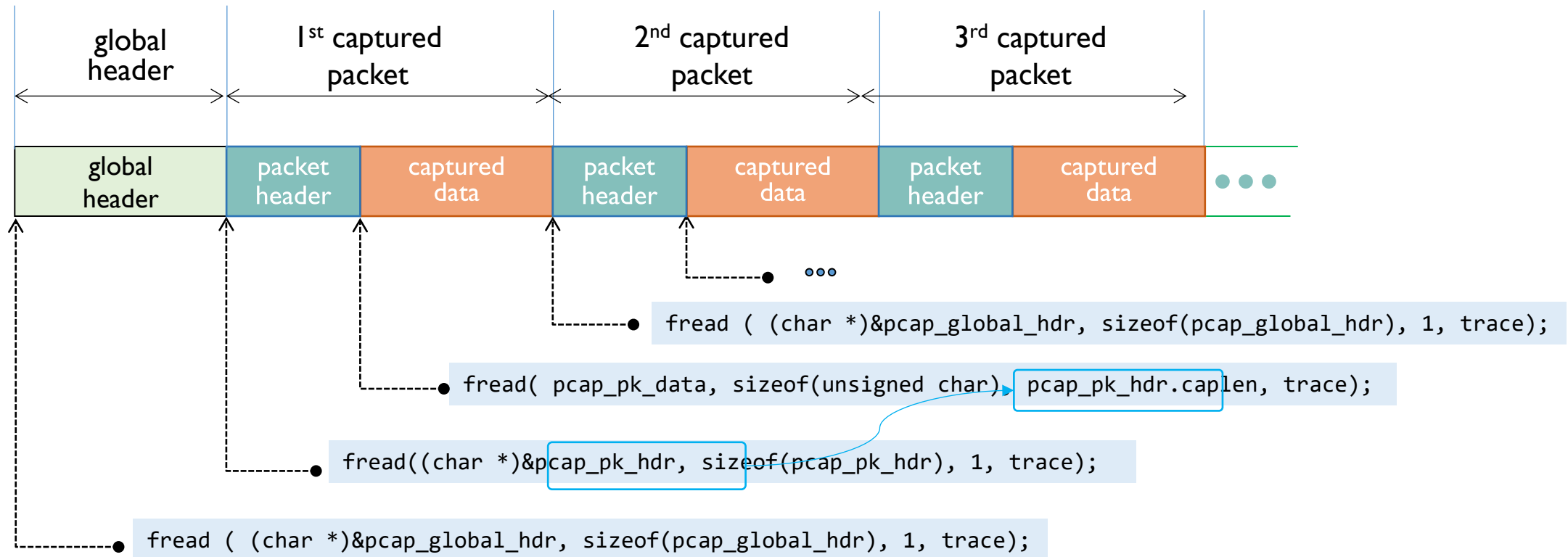
```
#define ntohs16(p)    ( (unsigned short)*((unsigned char *) (p)+0)<<8 | \
                        (unsigned short)*((unsigned char *) (p)+1)<<0 )
```

- macro to read a value with 4 bytes

```
#define ntohs32(p)    ( (unsigned short)*((unsigned char *) (p)+0)<<24 | \
                        (unsigned short)*((unsigned char *) (p)+1)<<16 | \
                        (unsigned short)*((unsigned char *) (p)+2)<<8  | \
                        (unsigned short)*((unsigned char *) (p)+3)<<0  )
```


Captured File Analysis – Data Structure

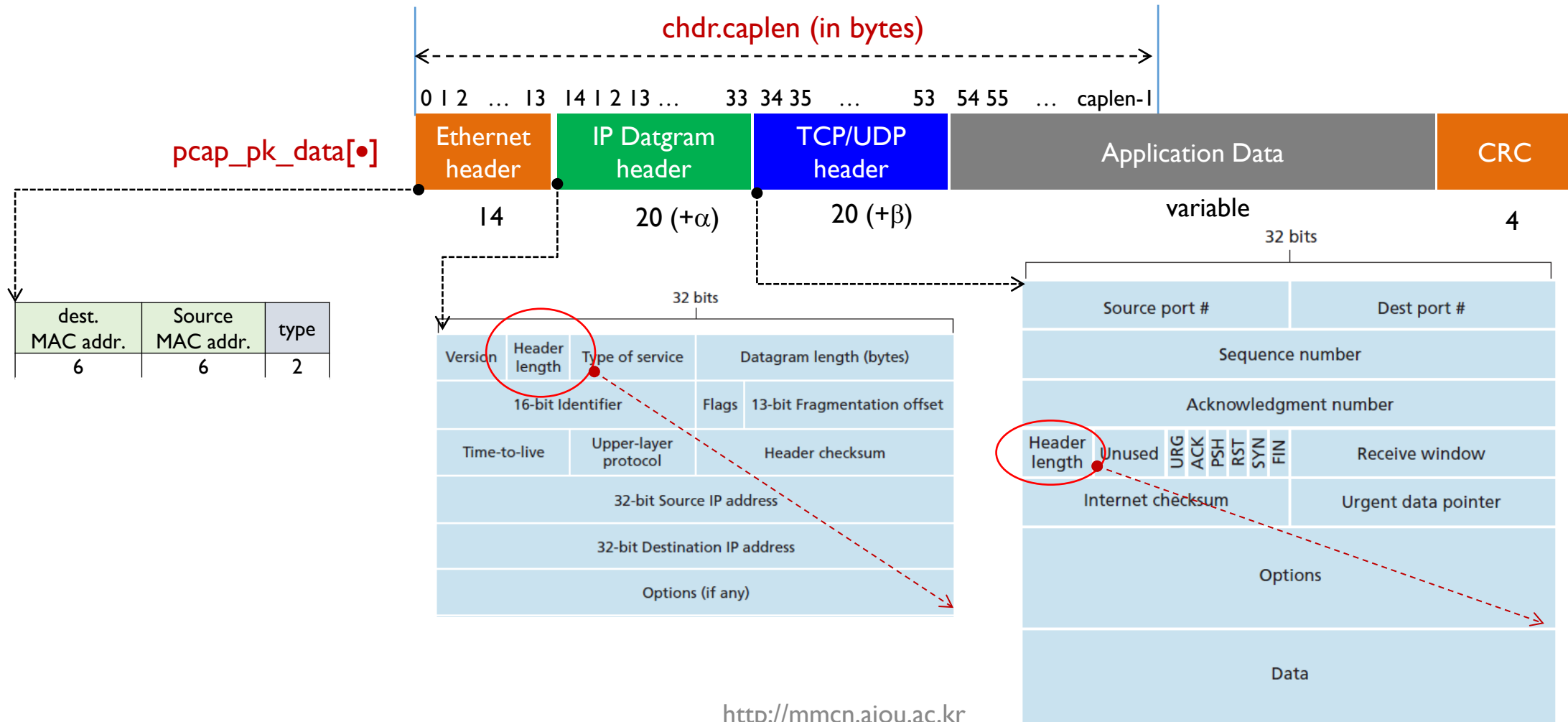
❖ Example – data structure



```
struct pcap_file_hdr    fhdr;  
struct pcap_pkt_hdr     chdr;
```

Captured File Analysis – Data Structure

❖ Example – buffer (packet data)



Captured File Analysis – Headers

❖ Headers

```
#include <stdio.h>
#include <stdlib.h>
#include <pcap.h>

#define ETHertype_IP      0x0800 // IP Type
#define ETH_II_HSIZE      14     // EthernetII 헤더 크기
#define IP_HSIZE          20     // IP 헤더 크기 (옵션 없음)
#define IP_PROTO_TCP      6      // TCP
#define IP_PROTO_UDP      17     // UDP

/* ethernet header */
struct ethernet_header {
    uint8_t dest_mac[6]; // 목적지 MAC 주소
    uint8_t src_mac[6];  // 소스 MAC 주소
    uint16_t eth_type;    // Ethernet 타입
};

// UDP Header
struct udp_header {
    uint16_t src_port; // 소스 포트 번호
    uint16_t dst_port; // 목적지 포트 번호
    uint16_t length;   // UDP 데이터그램 크기
    uint16_t checksum; // 체크섬
};
```

```
// IPv4 Header
struct ip_header {
    uint8_t ihl : 4; // IP 헤더 길이
    uint8_t version : 4; // IP 버전
    uint8_t tos; // 서비스 타입
    uint16_t tot_len; // 전체 길이
    uint16_t id; // 식별자
    uint16_t frag_off; // 플래그 + 오프셋
    uint8_t ttl; // TTL
    uint8_t protocol; // 프로토콜 (TCP, UDP 등)
    uint16_t checksum; // 체크섬
    uint32_t saddr; // 소스 IP 주소
    uint32_t daddr; // 목적지 IP 주소
};

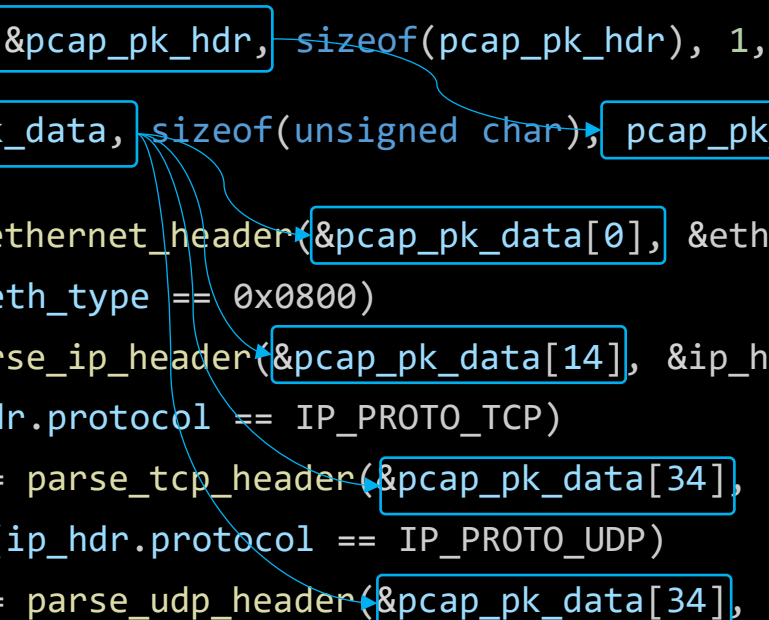
// TCP Header
struct tcp_header {
    uint16_t src_port; // 소스 포트 번호
    uint16_t dst_port; // 목적지 포트 번호
    uint32_t seq_num; // 시퀀스 번호
    uint32_t ack_num; // 응답 번호
    uint16_t hlen_flags; // 헤더길이(4비트) + Unused(6비트) + 플래그(6비트)
    uint16_t window; // 윈도우 크기
    uint16_t checksum; // 체크섬
    uint16_t urg_ptr; // 긴급포인터
};

struct ethernet_header eth_hdr;
struct ip_header ip_hdr;
struct tcp_header tcp_hdr;
struct udp_header udp_hdr;
```

Captured File Analysis – Main

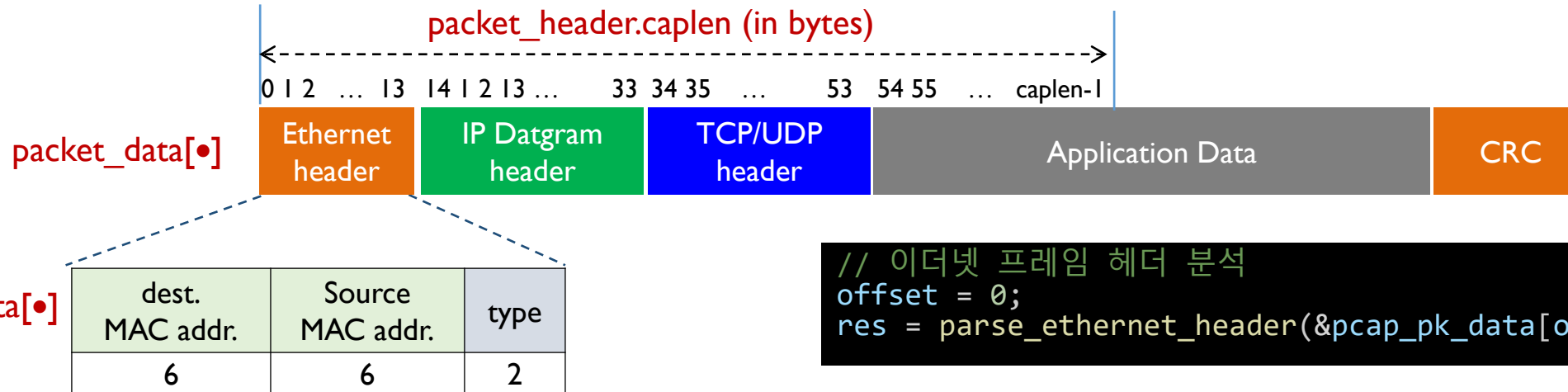
❖ Main Function

```
1 void main(int argc, char** argv)
2 {
3     fin = fopen("ccc.pcap", "rb");    // PCAP 파일 열기
4     fread((char*)&pcap_global_hdr, sizeof(pcap_global_hdr), 1, fin); // 글로벌 헤더 읽기
5
6     while (1) {
7         fread((char*)&pcap_pk_hdr, sizeof(pcap_pk_hdr), 1, fin); // 패킷 헤더 읽기
8
9         fread(pcap_pk_data, sizeof(unsigned char), pcap_pk_hdr.caplen, fin); // 패킷 데이터 읽기
10
11         res = parse_ethernet_header(&pcap_pk_data[0], &eth_hdr); // 이더넷 프레임 헤더 분석
12         if (eth_hdr.eth_type == 0x0800)
13             res = parse_ip_header(&pcap_pk_data[14], &ip_hdr); // IP 헤더 분석
14             if (ip_hdr.protocol == IP_PROTO_TCP)
15                 res = parse_tcp_header(&pcap_pk_data[34], &tcp_hdr); // TCP 헤더 분석
16             else if (ip_hdr.protocol == IP_PROTO_UDP)
17                 res = parse_udp_header(&pcap_pk_data[34], &udp_hdr); // UDP 헤더 분석
18         }
19         fclose(fin);
20 }
```



Captured File Analysis – Ethernet Header

❖ Ethernet Header

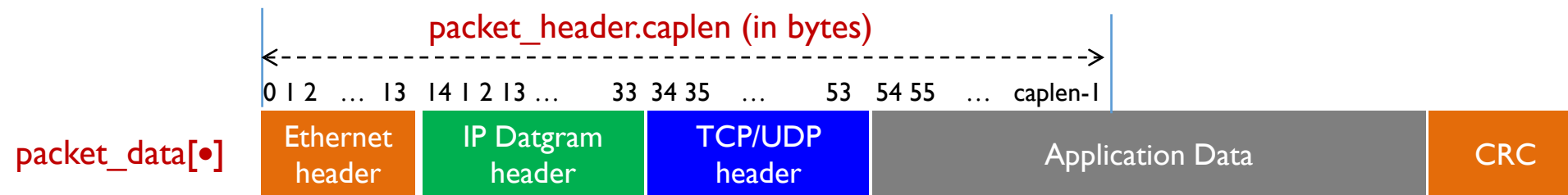


```
// 이더넷 프레임 헤더 분석
offset = 0;
res = parse_ethernet_header(&pcap_pk_data[offset], &eth_hdr);
```

```
1 int parse_ethernet_header(unsigned char* data, struct ethernet_header *eth_hdr)
2 {
3     for (int i = 0; i < 6; i++)
4         eth_hdr->dest_mac[i] = data[i];        // 발신지 MAC 주소
5
6     for (i = 0; i < 6; i++)
7         eth_hdr->src_mac[i] = data[i+6];      // 소스 MAC 주소
8
9     eth_hdr->eth_type = ntohs(&data[12]); // Type 필드
10
11     return 0;
12 }
```

Captured File Analysis – IP Header (1/2)

❖ IP Header



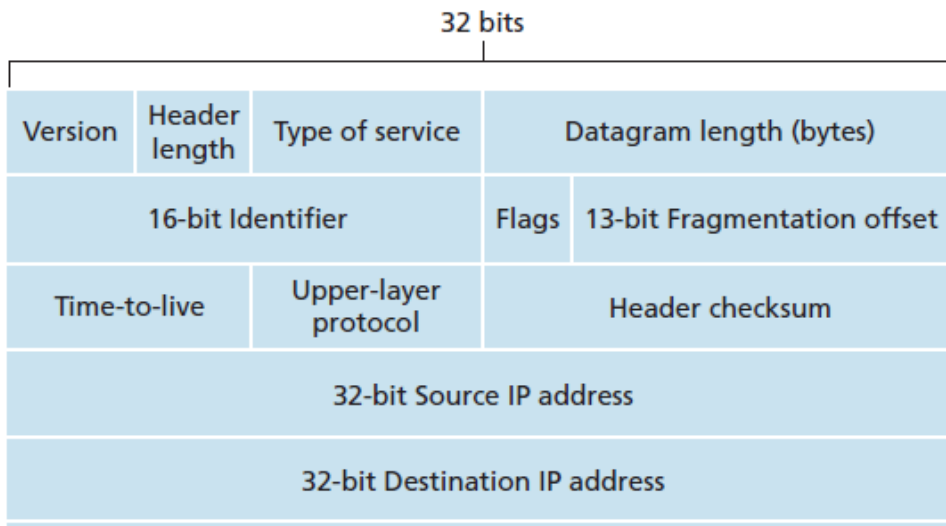
32 bits				
Version	Header length	Type of service	Datagram length (bytes)	
16-bit Identifier			Flags	13-bit Fragmentation offset
Time-to-live		Upper-layer protocol	Header checksum	
32-bit Source IP address				
32-bit Destination IP address				

```
// IP 헤더 분석
offset += ETH_II_HSIZE;
res = parse_ip_header(&pcap_pk_data[offset], &ip_hdr);
```

Captured File Analysis – IP Header (2/2)

❖ IP Header

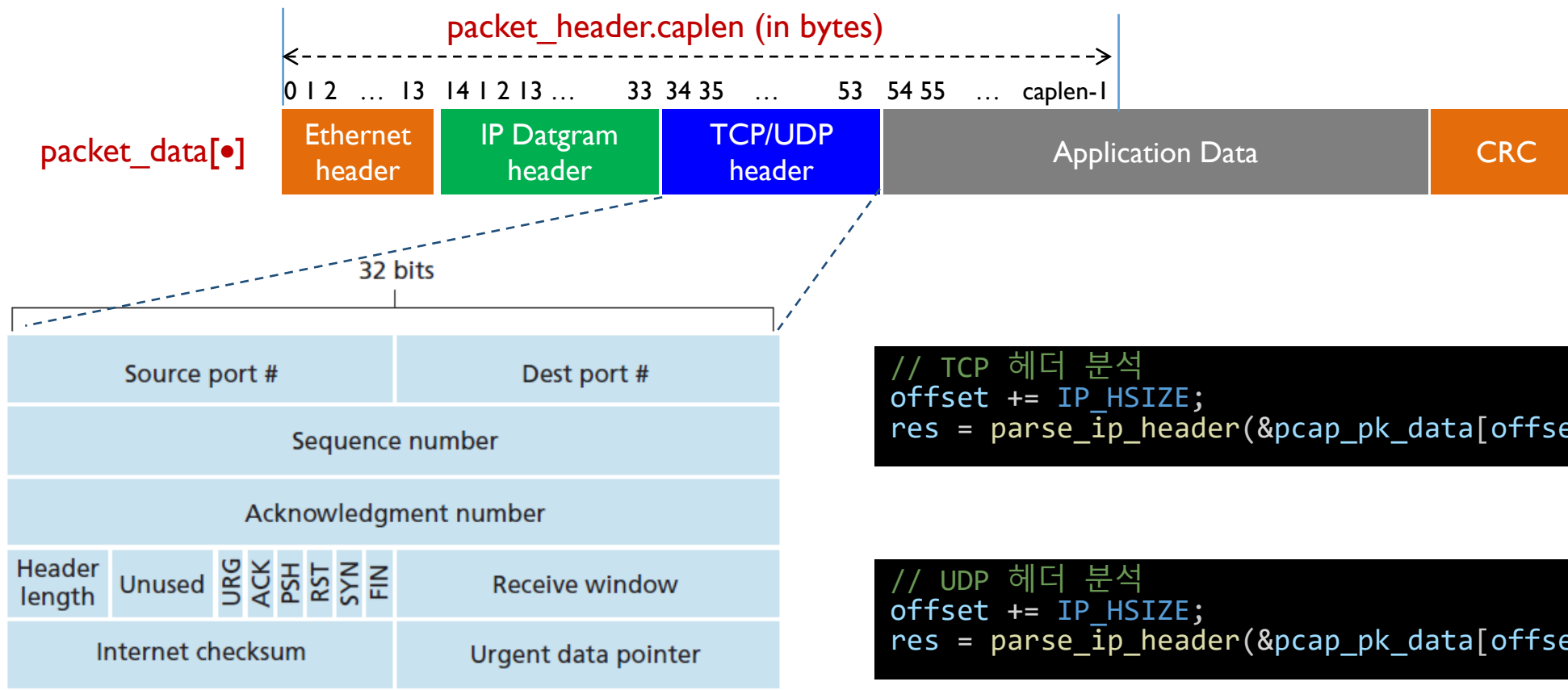
```
1 int parse_ip_header(unsigned char* data, struct ip_header *ip_hdr)
2 {
3     ip_hdr->version    = data[0] >> 4;    // IP version
4     ip_hdr->ihl        = data[0] & 0x0f;  // IP header length
5     ip_hdr->protocol    = data[9];        // protocol above IP
6
7     // 실습: 여기에 다른 IP 헤더 필드 정보 추가
8
9     return 0;
10 }
```



```
// IPv4 Header
struct ip_header {
    uint8_t  ihl : 4;    // IP 헤더 길이
    uint8_t  version : 4; // IP 버전
    uint8_t  tos;        // 서비스 타입
    uint16_t tot_len;    // 전체 길이
    uint16_t id;         // 식별자
    uint16_t frag_off;   // 플래그 + 오프셋
    uint8_t  ttl;        // TTL
    uint8_t  protocol;   // 프로토콜 (TCP, UDP 등)
    uint16_t checksum;   // 체크섬
    uint32_t saddr;      // 소스 IP 주소
    uint32_t daddr;      // 목적지 IP 주소
};
```

Captured File Analysis – TCP/UDP Header

❖ TCP/UDP Header



Captured File Analysis – Run

❖ Creation of a PCAP File

The image shows three overlapping screenshots of the Wireshark 'Capture Options' dialog box, illustrating the steps to create a PCAP file. The dialog box has three tabs: 'Input' (입력), 'Output' (출력), and 'Options' (옵션).

- Input Tab (입력):** The 'Capture Interface' (캡처 인터페이스 지정) is set to 'Wireshark - Loopback traffic capture' (Wireshark - 루프백 트래픽 캡처). The 'Promiscuous' (잡음) checkbox is checked, and the 'Filter' (필터) is set to 'ip'.
- Output Tab (출력):** The 'Output File' (출력 파일) is set to 'C:\Users\User\SynologyDrive\Bhroh\WVSPGM\WCaptureFileAnalysis\ccc.pcap'. The 'Output Format' (출력 형식) is set to 'pcapng'.
- Options Tab (옵션):** The 'Capture Size' (캡처 크기) is set to '500' packets. The 'Capture Filter' (캡처 필터) is set to 'ip'. The 'Capture Filter' (캡처 필터) is set to 'ip'.

Annotations in Korean:

- 캡처 인터페이스 지정 (Specify capture interface)
- Promiscuous 지정 (잡음 지정) (Promiscuous mode specified)
- IP 패킷만 캡처 (Capture only IP packets)
- 캐처후 저장할 파일 지정 (Specify file to save after capture)
- 500개 캡처후 파일 자동 저장 (Automatic saving of file after 500 captures)
- 시작 (Start)



Captured File Analysis – Run

❖ Run

```
#total number of packets : 100  
IP packets: 100  
non-IP packets: 0  
TCP packets: 100  
UDP packets: 0
```



ON-SPOT PRACTICE

Task 1: IP 헤더 분석 함수 완성

❖ 다음 IP 헤더 분석 함수에 다른 필드들이 반영되도록 수정

```
1 int parse_ip_header(unsigned char* data, struct ip_header *ip_hdr)
2 {
3     ip_hdr->version      = data[0] >> 4;      // IP version
4     ip_hdr->ihl           = data[0] & 0x0f;    // IP header length
5     ip_hdr->protocol      = data[9];           // protocol above IP
6     // 실습: 여기에 다른 IP 헤더 필드 정보 추가
7
8     return 0;
9 }
```

32 bits				
Version	Header length	Type of service	Datagram length (bytes)	
16-bit Identifier			Flags	13-bit Fragmentation offset
Time-to-live	Upper-layer protocol		Header checksum	
32-bit Source IP address				
32-bit Destination IP address				

```
// IPv4 Header
struct ip_header {
    uint8_t  ihl : 4;      // IP 헤더 길이
    uint8_t  version : 4;  // IP 버전
    uint8_t  tos;          // 서비스 타입
    uint16_t tot_len;      // 전체 길이
    uint16_t id;           // 식별자
    uint16_t frag_off;     // 플래그 + 오프셋
    uint8_t  ttl;          // TTL
    uint8_t  protocol;     // 프로토콜 (TCP, UDP 등)
    uint16_t checksum;     // 체크섬
    uint32_t saddr;        // 소스 IP 주소
    uint32_t daddr;        // 목적지 IP 주소
};
```

Task 2: Wireshark Packet List Pane 작성

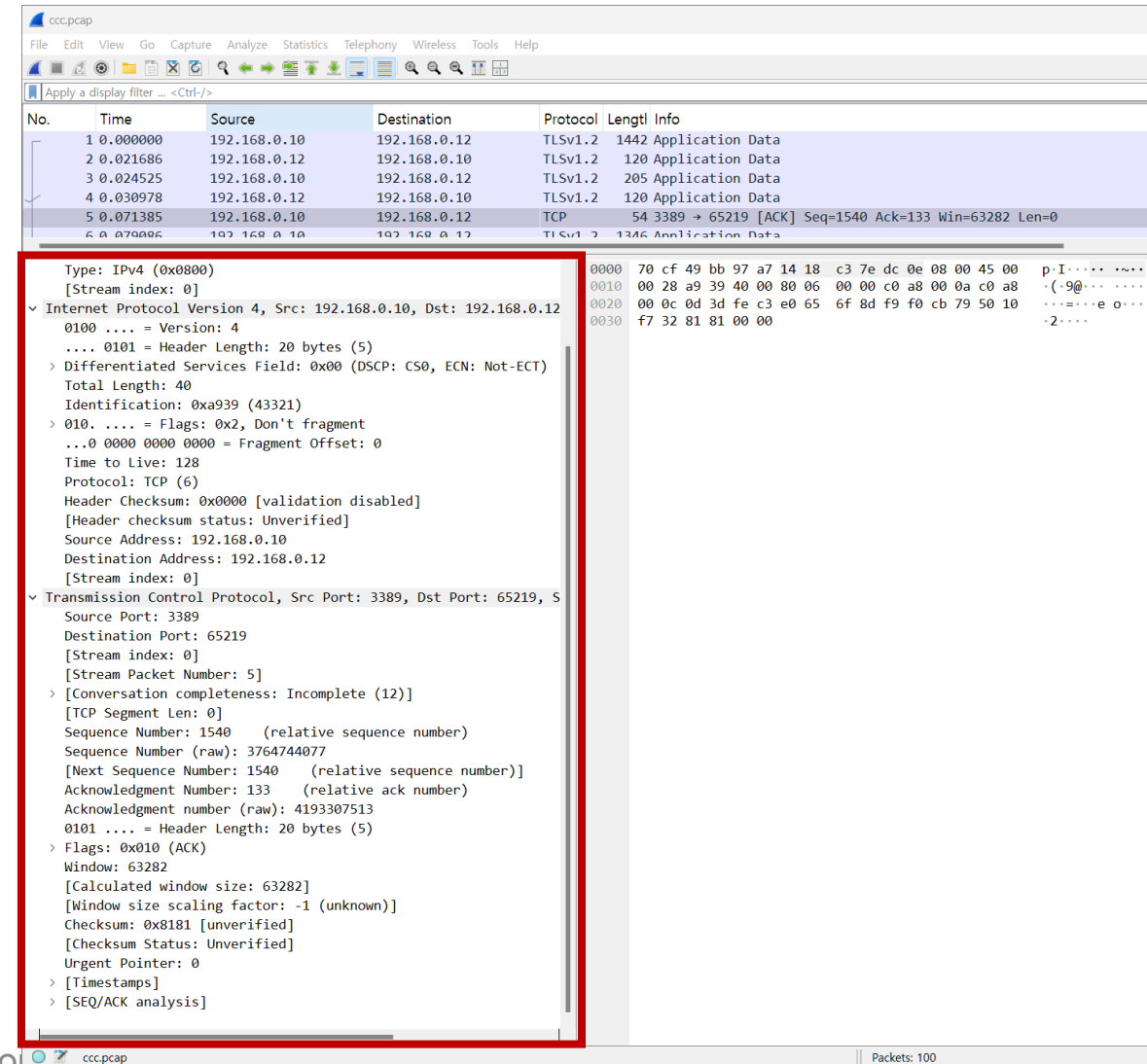
- ❖ 아래의 매 패킷 데이터를 분석하여 Wireshark의 Packet List Pane과 유사한 정보가 나오도록 프로그램 수정 (//실습 I 부분)

No.	Time	Source	Destination	Protocol	Length	Info
5	0.071385	192.168.0.10	192.168.0.12	TCP	54	3389 → 65219 [ACK] Seq=1540 Ack=133 Win=63282 Len=0
6	0.079086	192.168.0.10	192.168.0.12	TLSv1.2	1346	Application Data
7	0.094481	192.168.0.12	192.168.0.10	TLSv1.2	120	Application Data
8	0.113623	192.168.0.10	192.168.0.12	TLSv1.2	1699	Application Data
9	0.113685	192.168.0.10	192.168.0.12	TLSv1.2	1699	Application Data
10	0.113710	192.168.0.10	192.168.0.12	TLSv1.2	1699	Application Data

- Protocol은 TCP, UDP 또는 ETC로 표현
- Info에는
 - TCP인 경우: source port → destination port : [Flag 정보]
 - 플래그정보는 UAPRSF 에서 비트값이 0인 부분은 0, 1인 부분은 해당 플래그 표현
 - 예를들어, ACK, SYN이 1로 된경우는 [Flag: 0A00S0]과 같이 표현
 - UDP인 경우: source port → destination port 만 표현
 - ETC인 경우: IP의 protocol 필드 값 표현 (예: protocol = 값)

Task 3: Wireshark Packet Details Pane 작성

- ❖ 패킷 데이터 하나씩 읽어서
 - ethernet, ip, tcp/udp 헤더 필드들을 나타냄
- ❖ 입력 받고
 - 다음 패킷으로 진행
- ❖ 입력값이 0 이면
 - 끝까지 진행
 - 이때, Task 2의 내역이 출력됨



Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN



Mobile **Multimedia Convergence** Network Lab.

Homepage: <http://mmcn.ajou.ac.kr>

facebook: <http://www.facebook.com/#!/groups/190702010947314>

